软件系统综合课程设计

——系统测试及软件性能



什么是系统测试

系统测试是将经过集成测试的软件，作为计算机系统的一个部分。与系统中其他部分结合起来，在实际运行环境中对整个软硬件系统进行一系列测试，以发现软件中潜在的问题。

是对整个软件系统进行测试的过程，目的是验证系统是否满足规格要求，并且是否符合用户需求。

测试的是软件，以及运行依赖的硬件和接口、操作系统，其他支撑软件以及相关数据等。

什么是系统测试

系统测试包含了多种测试活动，主要分为功能性测试和非功能性测试两大类。



功能性测试通常检查软件功能的需求是否与用户的需求一致。包括：功能测试、用户界面测试、安装/卸载测试、可适用性测试；

非功能性测试主要检查软件的性能、安全性、健壮性。包括：性能测试、压力测试、兼容性测试、可靠性测试、安全测试、健壮性测试等等。

系统测试的数据

系统测试所用的数据应当尽可能地像真实数据一样精确和有代表性，也应当和真实数据的大小和复杂性相当。

如果测试数据很简单，不能反映软件实际使用时的真实情况，那么这样的系统测试就是浪费时间，没有意义，无法对软件进行有效的检验和测试。

要让测试数据与真实数据的大小和复杂性相当，一个简单的方法就是直接使用真实数据作为测试数据。

这当然是一种有效而便捷的做法，但有时出于信息安全、隐私保护等原因，而无法直接使用真实数据作为测试数据。

系统测试的数据

可以在真实数据的基础上，对信息进行加密或者对数据进行扭曲，在保持真实数据的精度、数据量、复杂度等特性的情况下，隐藏掉真实数据中的有效信息，让它们既可用于软件测试，又不会泄露有效信息。

在此基础上仍然有必要引入一些专门设计的测试数据，以利于有针对性的发现软件中可能存在的问题。在设计这些测试数据时，测试人员必须采用相应的测试设计技术，使得设计的数据真正有代表性和针对性，实现对软件系统足够充分地测试。

性能测试

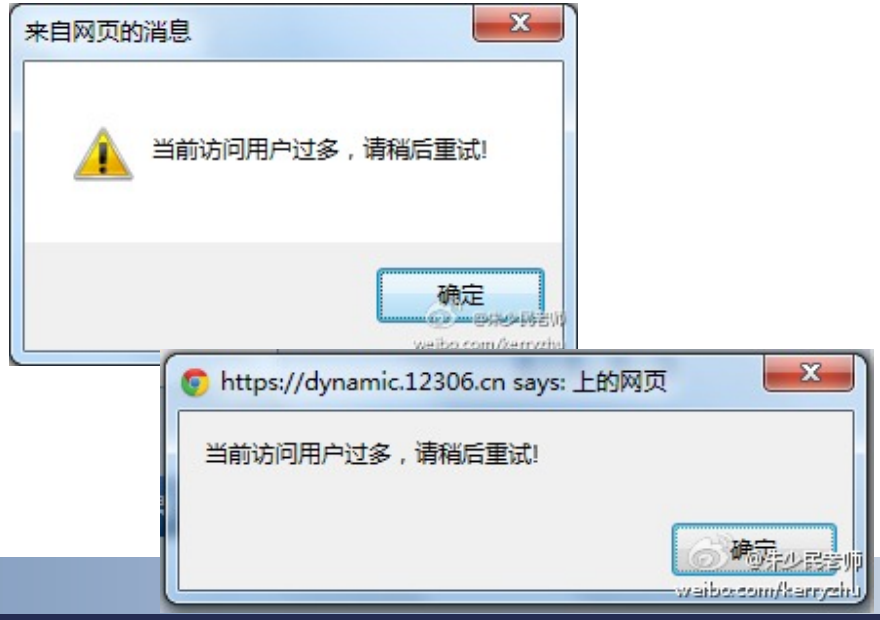
性能问题是常见的问题，特别是在今天SaaS时代、万物互联时代

压力测试、容量测试等也并入性能测试

性能测试一般只能通过工具来实现

性能测试与性能调优相辅相成，循环往复、不断提升性能





性能问题外部表现和内因

- 启动系统、打开页面越来越慢
- 查询数据，很长时间才显示列表
- 网络下载速度很低，如5k/s
- 资源耗尽，如CPU使用率达到100%
- 资源泄漏，如内存泄漏 ， 最终会导致资源耗尽
- 资源瓶颈，如线程、GDI、DB连接等资源变得稀缺

系统测试-性能测试

性能测试是通过自动化测试工具模拟许多正常、峰值以及异常负载条件来对系统的各项性能指标进行测试。

许多软件都有其特殊的性能或效率目标要求即在一定工作负荷和资源配置条件下，对响应时间、处理速度等特性有指标要求，

- 获取系统性能某些指标数据
- 为了验证系统是否达到用户提出的性能指标
- 发现系统中存在的性能瓶颈，优化系统的性能

性能测试

- 在实际的测试之中，功能特性方面的测试逐步地从系统测试中分离了出来，而将系统测试定义为非功能性测试，主要是针对负载测试、性能测试、容量测试、完全性测试、兼容性测试、可靠性测试等。
- 而在工程应用中，性能测试广义上往往包括了常规性能测试、负载测试、压力测试、容量测试、配置测试等。
- 但从狭义上讲，性能测试仅测试性能是否满足需求规格说明书的要求。

背景及其分析

- 系统性能的改善是测试、调整、再测试、再调整、……一个持续改进的过程——性能调优
- 性能调优需要借助负载测试方法的帮助
- 负载测试可以看作是性能测试所采用的一种技术
- 压力测试可以被看作是负载测试的一种，即高负载下的负载测试
- 容量测试也采用负载测试技术来实现
- 稳定性测试：检验长时间高负载下的稳定性

性能测试分类

- 负载测试是通过模拟实际软件系统所承受的负载条件、改变系统负载大小和负载方式来发现系统中所存在的问题
- 压力测试是在强负载情况下（如大数据量、大量并发用户连接等）稳定性进行测试，查看应用系统在峰值（瞬间使用高峰）使用情况下的行为表现，更有效地发现系统稳定性的隐患和系统在负载峰值的条件下功能隐患等，确认系统是否具有良好的容错能力和可恢复能力。 为了获取或验证系统性能指标、或为了发现性能瓶颈而进行的测试

性能测试



所谓性能测试就是通过性能测试工具模拟正常、峰值及异常负载状态下对系统的各项性能指标进行测试的活动。性能测试能够验证软件系统是否达到了用户期望的性能需求，同时也可以发现系统中可能存在的性能瓶颈及缺陷，从而优化系统的性能。

不同角度的性能测试-用户角度

① 用户视角

响应时间　处理速度

② 系统视角

延迟(包括网络延迟、数据库延迟等)　　系统资源使用状况

③ 开发者视角

代码实现的执行效率　　数据库实现的执行效率

不同视角的系统性能-开发者角度

开发人员关心的问题	问题所属层次
（1）架构设计是否合理	系统架构
（2）数据库设计是否存在问题	数据库设计
（3）代码是否存在性能方面的问题	代码
（4）系统中是否有不合理的内存使用方式	代码
（5）系统中是否存在不合理的线程同步方式	设计与代码
（6）系统中是否存在不合理的资源竞争	设计与代码

指标分类	指标分类中文	常用性能指标
Load Size	负载大小	负载数据量(Load size)
Min	最小值	连接时间(Connect Time)
Max	最大值	发送时间(Sent Time)
Average	平均值	处理时间(Process Time)
Current value	当前值	一轮来回时间(Round time)
Transactions Per Second	每秒事务数(TPS)	平均事务响应时间
Successful Transactions Per Second	每秒成功事务数	每秒事务总数
Failed Transactions Per Second	每秒失败事务数	每秒点击次数图
Rounds Per Second	每秒轮次	每秒 HTTP 响应数
Successful Rounds Per Second	每秒成功轮次	每秒下载页面数
Failed Rounds Per Second	每秒失败轮次	每秒重试次数
Throughput (Bytes Per Second)	吞吐量(字节/秒)	连接数
Response Data Size	响应数据大小	每秒连接数
Round Time	轮次时间	每秒 SSL 连接数
Transaction Time	事务时间	页面下载时间
Connect Time	连接时间	第一次缓冲时间
Send Time	发送时间	已下载组件大小

性能测试指标和类型

指标分类	指标分类中文
Response Time	响应时间
Process Time	处理时间
Rounds	轮次总数
Successful Rounds	成功轮次总数
Failed Rounds	失败轮次总数
Transactions	事务总数
Successful Transactions	成功事务总数
Failed Transactions	失败事务总数
Attempted Connections	尝试连接总数
Successful Connections	成功连接总数
Failed Connections	失败连接总数
Responses	响应总数

性能测试目标

1、响应时间

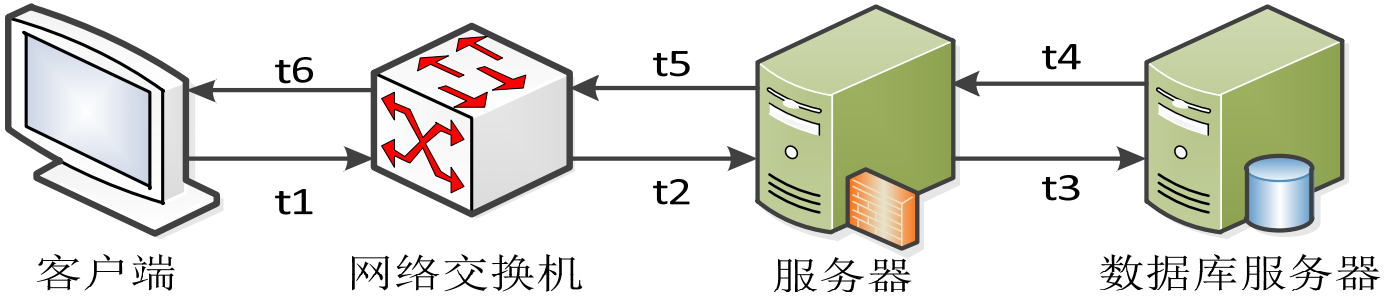
响应时间（Response Time）指系统对用户请求作出响应所需要的时间。

这个时间是指用户从软件客户端发出请求到用户接收到返回数据的整个过程所需要的时间，包括各种中间件（如服务器、数据库等）的处理时间。

响应时间越短，表明软件的响应速度越快，性能越好。但是响应时间需要与用户的具体需求相结合，例如火车订票查询功能响应时间一般2s内就可以完成，而在网站下载电影时，几分钟完成下载的网站就已经很快了。

响应时间

对用户来说，当用户单击一个按钮、发出一条指令或是在Web页面上单击一个链接，从用户单击开始到应用系统把本次操作的结果以用户能察觉的方式展示出来，这个过程所消耗的时间就是用户对软件性能的直观印象



响应时间=网络响应时间 应用程序响应时间

系统的响应时间为t1+t2+t3+t4+t5+t6

响应时间范围 2/5/10

在2秒之内给客户响应会被用户认为是“非常有吸引力的”，在5秒之内响应客户会被认为是“比较不错的”，而10秒是客户能接受的等待响应的上限，如果超过10秒还没有得到响应，那么大多数用户不会继续等待，而是放弃操作。

（1） 一般响应时间

指完成一些常规操作如增加、删除、修改、查询等需要的响应时间。

测试时可以选择一些事务的实例来执行即可

（2）特殊响应时间

需要指完成一些特殊的操作，需要的响应时间，分别定义。

测试时也需要单独设计并逐项测试

性能测试时间

（1）连接时间:客户机和服务器建立连接的时间

（2）发送时间:客户机向服务器发送服务请求的时间

（3）处理时间:服务器处理客户机请求所需要的时间

（4）接收时间:客户机接收服务器响应数据的时间

（5）呈现时间:客户机处理接收数据并呈现需要的时间



2、吞吐量

吞吐量（Throughput）是指单位时间内系统能够完成的工作量，它衡量的是软件系统服务器的处理能力。

吞吐量的度量单位可以是请求数/秒、页面数/秒、访问人数/天、处理业务数/小时等。

吞吐量是软件系统衡量自身负载能力的一个很重要的指标，吞吐量越大，系统单位时间内处理的数据就越多，系统的负载能力就越强。

3、并发用户数

并发用户数是指同一时间请求和访问的用户数量。

并发用户数量越大，对系统的性能影响越大，并发用户数量较大可能会导致系统响应变慢、系统不稳定等。软件系统在设计时必须要考虑并发访问的情况，测试工程师在进行性能测试时也必须进行并发访问的测试。

4、TPS

TPS是指系统每秒钟能够处理的事务和交易的数量，

它是衡量系统处理能力的重要指标。 次/单位时间

5、点击率

点击率是指用户每秒向Web服务器提交的HTTP请求数，这个指标是Web应用特有的一个性能指标，通过点击率可以评估用户产生的负载量，并且可以判断系统是否稳定。

点击率只是一个参考指标，帮助衡量Web服务器的性能。

6、资源利用率

资源利用率是指软件对系统资源的使用情况，包括CPU利用率、内存利用率、磁盘利用率等，资源利用率是分析软件性能瓶颈的重要参数。

性能问题在系统内部表现为资源使用耗尽或使用率过高,在外部表现为系统响应很慢

(1)资源耗尽,如 CPU 使用率达到 100%。

(2)资源泄漏,如内存泄漏,最终会导致资源耗尽

(3)资源瓶颈,如线程、GD1、DB 连接等资源变得稀缺,

性能测试中的通用指标

（1）ProcessorTime:

指服务器CPU占用率，一般平均达到70%时，服务就接近饱和;

（2）Memory Available Mbyte :

可用内存数如果测试时发现可用内存有变化也要注意，如果是内存泄露则比较严重;

（3）Physicsdisk Time :

物理磁盘读写时间;

一些常见的性能问题

资源泄漏，包括内存泄漏

资源瓶颈，内部资源（线程、放入池的对象）变得稀缺

CPU使用率达到100%、系统被锁定等

线程死锁、线程阻塞等

数据库连接成为性能瓶颈

查询速度慢或列表效率低

受外部系统影响越来越大

如何确定性能需求

只有具备了清楚而量化的性能指标，性能测试才能开始实施

- 最终用户的体验，如2-5-10原则
- 商业需求，如“比竞争对手的产品好”
- 技术需求，如CPU使用率不超过70%
- 标准要求

响应时间是用户的关注点，容量和数据吞吐量是（产品市场团队）业务处理方面的关注点，而系统资源占用率是开发团队的技术关注点

性能测试类型

- 性能验证测试，验证事先已定义的系统性能指标、系统能否满足系统的性能需求、这种测试的前提是事先能够明确系统的性能指标。
- 性能基准测试，在系统标准配置下获得有关的性能指标数据，作为将来性能改进的基准线，开发一个全新的系统时，可能无法确定系统的性能指标，这种情况下需要获得产品第一版本的性能指标，之后的版本就可以根据第一个版本的数据提出具体改进的指标。

性能测试类型

- 性能规划测试，在多种特定的环境下，获得不同配置的系统的性能指标，从而决定在系统部署时采用什么样的软、硬件配置
- 容量测试可以看作性能的测试一种，因为系统的容量可以看作是系统性能指标之一

  压力测试、负载测试

性能测试的细分类别

负载测试：

测试当负载逐渐增加时，系统各项性能指标的变化情况，从而可以明确在各种工作负载下系统的性能，指导系统的部署和应用

压力测试：

测试系统在当前软硬件环境下所能承受的最大负载并帮助找出系统瓶颈所在，以促进系统的性能改进

并发测试：

主要测试当有多个用户并发访问同一个应用、模块或者数据时是否会产生隐藏的并发问题，如内存泄漏线程锁、资源争用问题，几乎所有的性能测试都会涉及并发测试

性能测试的细分类别

稳定性测试（Endurance Test）：

目的：长时间运行（如24小时）检测内存泄漏、性能衰减\关键指标：资源利用率随时间的变化趋势

容量测试（**Capacity Test**）

目的：确定系统在可接受性能下的最大用户/数据承载量

应用：为扩容决策提供数据支持，如电梯最大承载量测算

可靠性测试（**Fault Tolerance Test**）

目的：验证系统故障后的自愈能力和业务连续性

方法：通过故障注入模拟异常场景

性能测试用例的设计流程

需求分析　　确定指标　　设计场景

需求分析

明确测试目标（如评估响应时间、吞吐量、资源利用率等）识别核心业务场景（如登录、支付、搜索等高频率或资源密集型操作）

指标定义

用户感知指标：响应时间（如95%请求≤2秒）、错误率（≤0.1%）系统能力指标：TPS（每秒事务数）、并发用户数、吞吐量

资源消耗指标：CPU利用率（≤75%）、内存占用（≤70%）、数据库连接池使用率

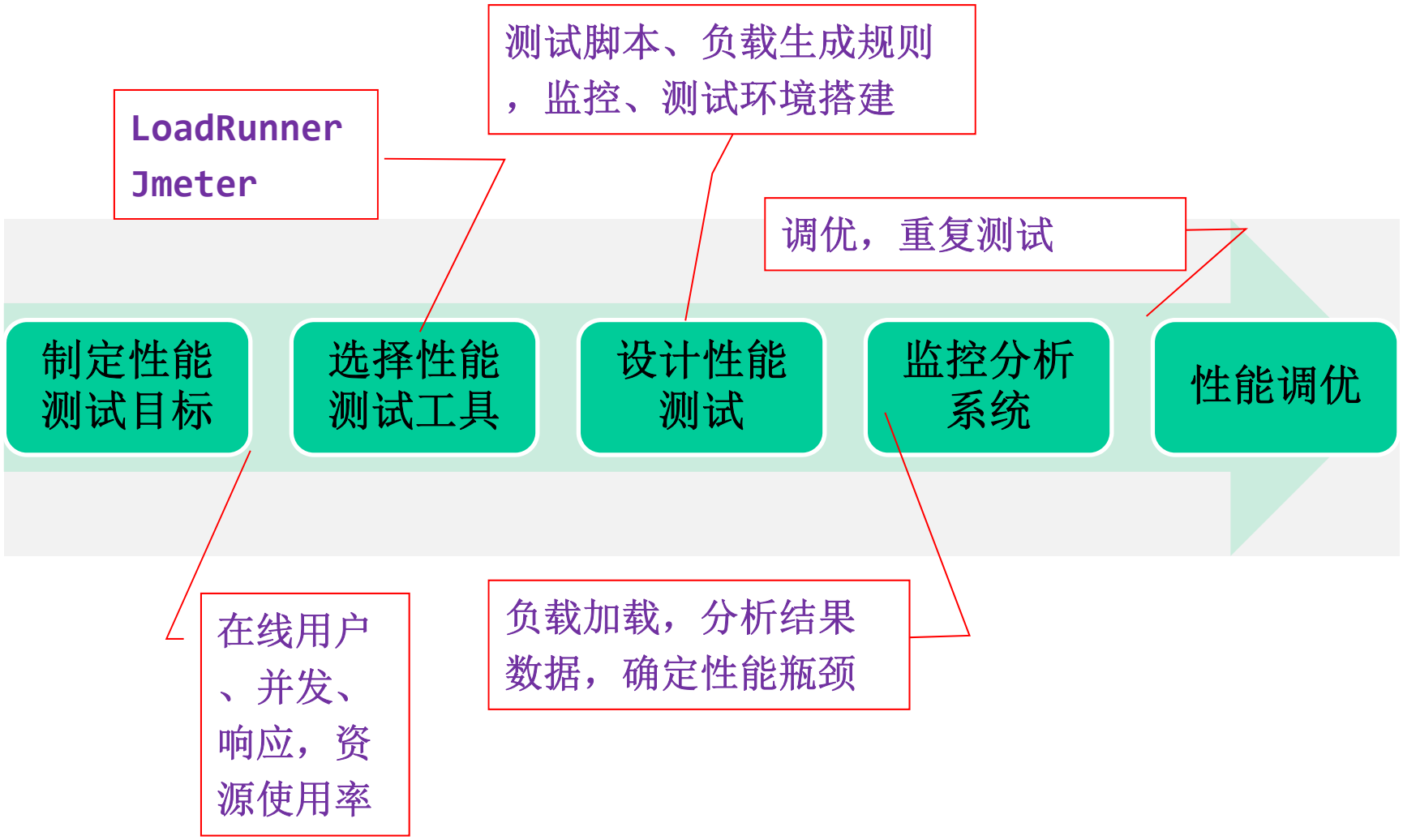
场景设计

单接口测试：验证独立接口性能（如登录接口）

组合业务测试：模拟完整业务流程（如电商下单→支付）

异常场景：超负载、网络波动、数据量激增等

性能测试流程


LoadRunner
Jmeter
测试脚本、负载生成规则，监控、测试环境搭建
调优，重复测试
制定性能测试目标
选择性能测试工具
设计性能测试
监控分析系统
性能调优
在线用户、并发、响应，资源使用率
负载加载，分析结果数据，确定性能瓶颈

性能测试与性能优化

性能测试的目的之一就是要通过测试，来发现性能问题，并进行系统性能的优化。

（1）对应用软件、中间件、数据库等的优化;一般而言对数据库的调优的效果要好于程序调优

（2）对服务器系统参数配置优化，

（3）升级客户端、服务器硬件、改善网络性能或路由等

在性能测试中往往要模拟很多个用户同时访问系统，精确记录响应时间，实时监控系统资源使用情况等，这些工作都很难靠手工来完成，所以性能测试需要用到多种测试工具

性能测试工具

性能测试工具要完成的工作可以分为三大类

（1）负载生成

（2）客户应用运行

（3）资源监控。

负载（Workload）

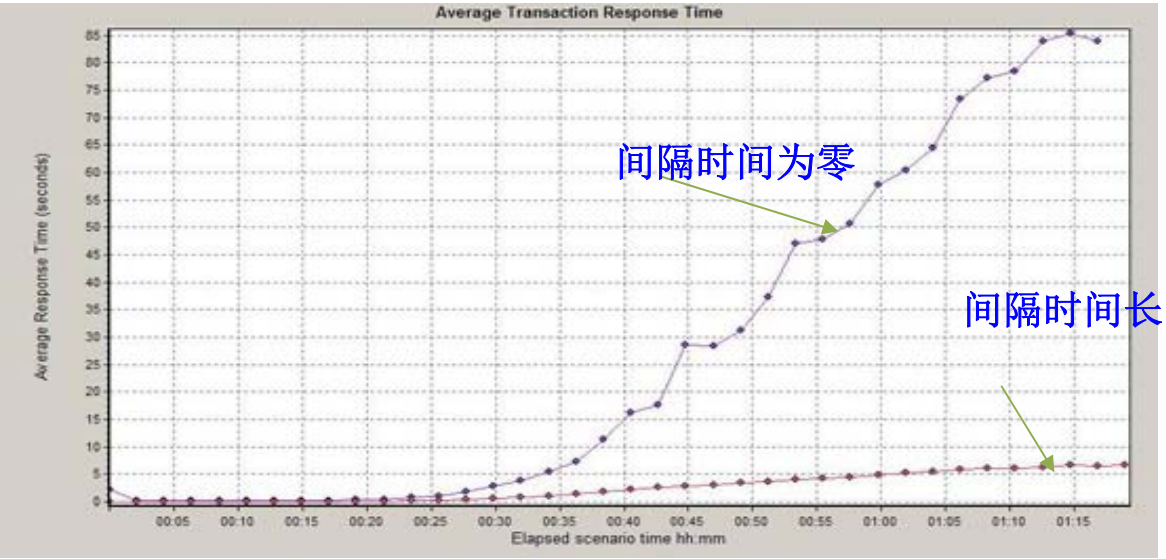
- 每次请求发送的数据量 (Request Per Second, **RPS**)
- 并发连接数 (Simultaneous Connections, SC)
- 思考时间（thinking time），用户发出请求之间的间隔时间

RPS + SC + Thinking Time = Concurrent users

$C=n\times L/T$：*n*为每日活跃用户数，*L*为单用户平均会话时长，T为统计时间段　C是平均并发用户数

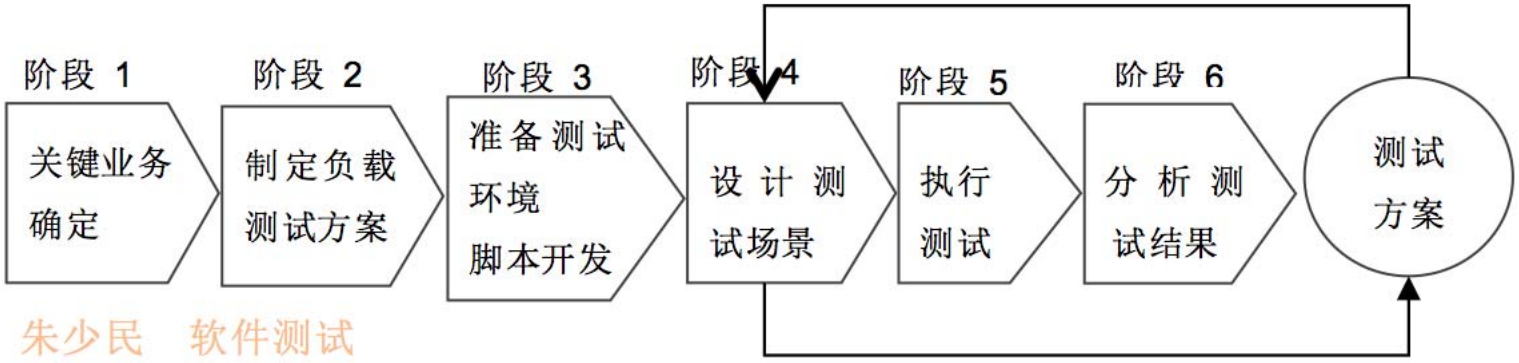
思考时间的影响

将系统置于相同的高负载下，将请求之间间隔时间设为零。这样服务器会立即超载，并形成越来越长的等候队列，直至内存耗尽



负载测试过程

- 确定所要模拟的角色及其对应的关键业务操作路径
- 确定输入/输出参数，制定负载测试方案
- 准备测试环境，并完成相应的测试脚本的开发
- 设计具体的测试场景，如负载水平、加载方式等
- 执行测试，监控输出参数，如数据吞吐量、响应时间、资源占有率等
- 对测试结果进行分析
- 结果不满意，需要调整测试场景，进入下一个循环



朱少民 软件测试

输入参数

负载测试是通过模拟用户的操作方式来考察系统的行为，所以人们肯定会问：如何模拟用户的行为？

1、并发用户数、并发连接数等。

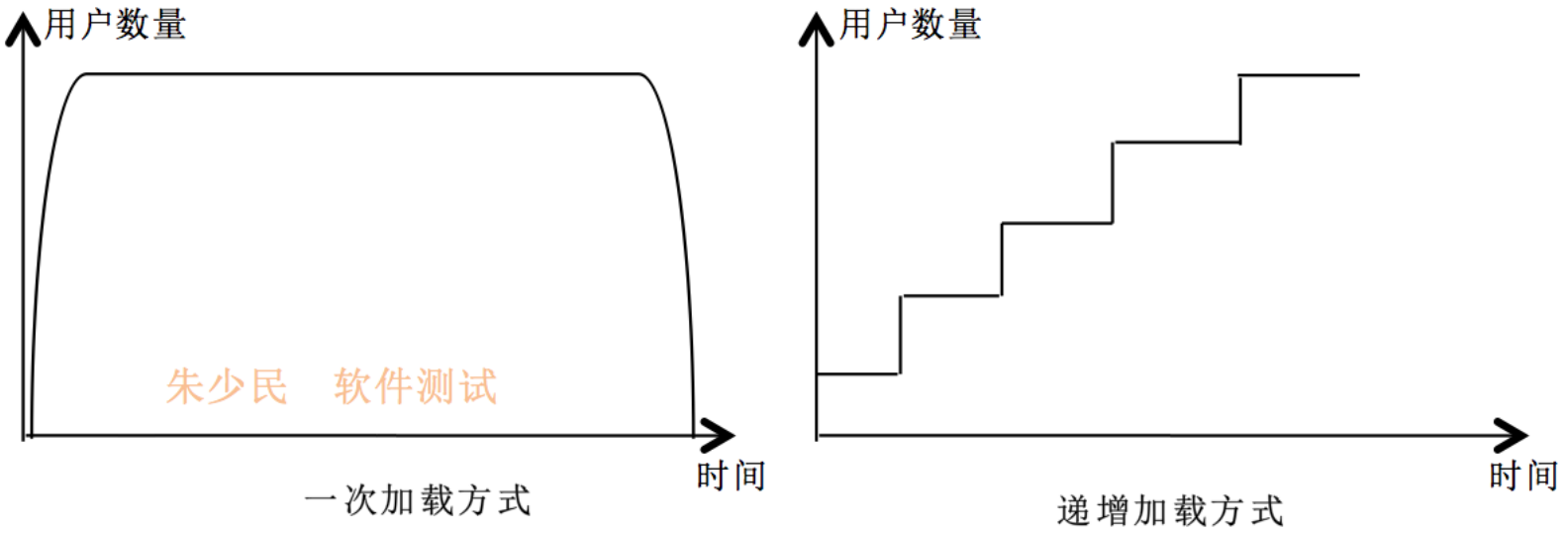
2、思考时间（think time），用户发出请求之间的间隔时间

3、加载的循环次数或持续时间

4、每次请求发送的数据量。

5、加载的方式或模式，如均匀加载、峰值交替加载等

加载模式

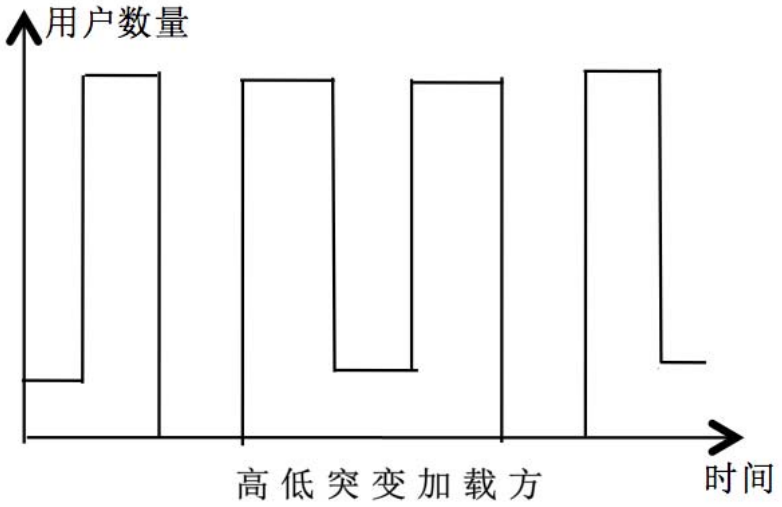


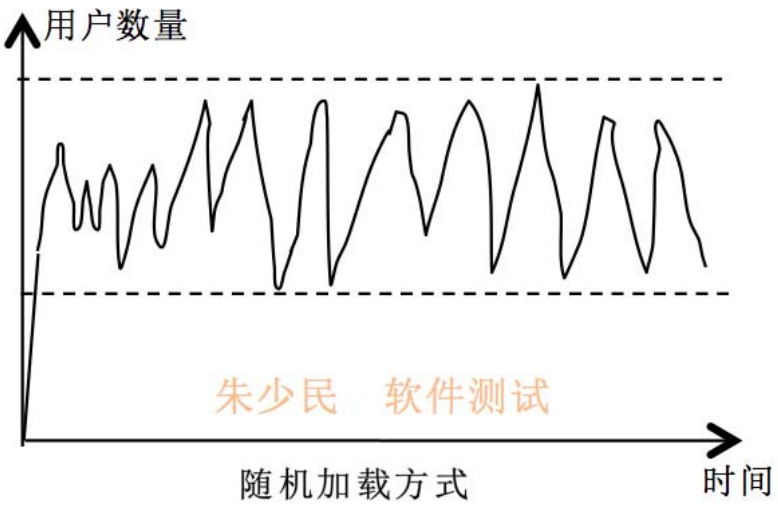
朱少民　软件测试

一次加载方式：一次性加载x数量用户，预定时间段内持续运行。主要测试瞬时高峰期压力。

递增或递减加载方式：用户规律逐渐加载，每几秒增加x用户，交错上升。主要用于发现性能拐点，确定负载基线，响应时间超出阈值，错误超出阈值。

加载模式



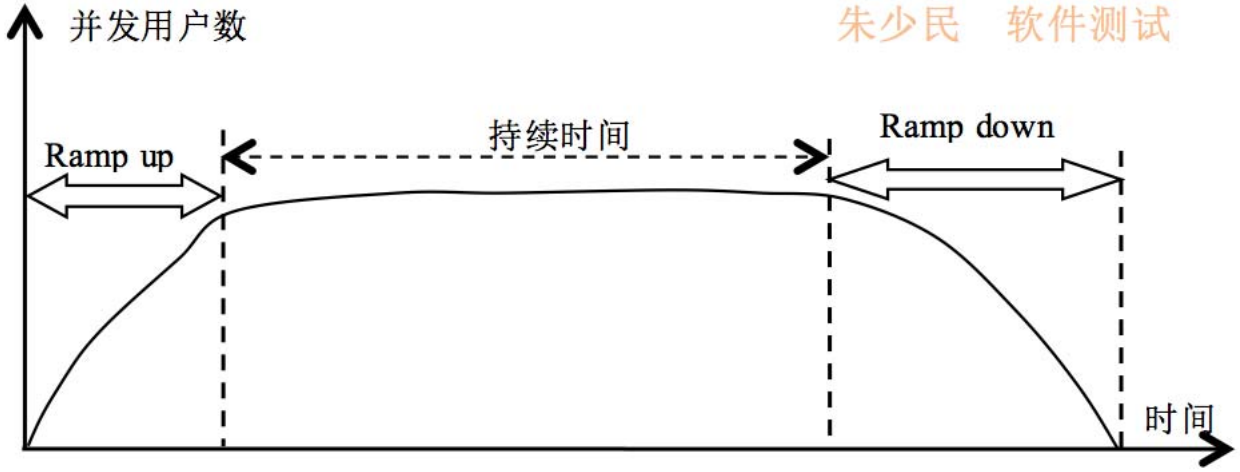


高低突变加载方式：某个时间用户数量很大，突然降低，一段时间，又突然加到很高，往复几次。主要用于发现资源释放，内存泄漏等问题。

随机加载方式:随机算法自动生成某个数量范围内变化，动态负载，模拟实际情况。主要用于模拟较长时间高位运行过程，非瞬时高峰期。

场景设置

在性能测试执行前，以什么样方式启动负载方式、如何持续进行负载测试直至负载测试结束，这个过程的负载大小和方式、负载启动和结束以及各种检查点、验证点等设计，被称为场景设置。



启动（Ramp up） 持续期间（Duration） 结束（Ramp down）

场景类型

- 可以分为静态和动态两部分。静态部分是指设置模拟用户生成器、用户数量、用户组等，动态部分主要指添加性能计数器、检查点、阀值等，从而获得负载测试过程中反回来馈的数据——系统运行的动态状态。
- 可以依据业务模式变化、随时间段变化来进行设置
- 也可分为手工场景 和面向目标的场景

负载测试执行

- 大量的虚拟用户要运行在多个客户端，并由控制器管理、代理（agent）驱动
- 负载测试的执行，需要针对不同维度的变化进行，包括时间维、负载维和系统维
- 监控、详细的记录和适当的分析是十分重要的

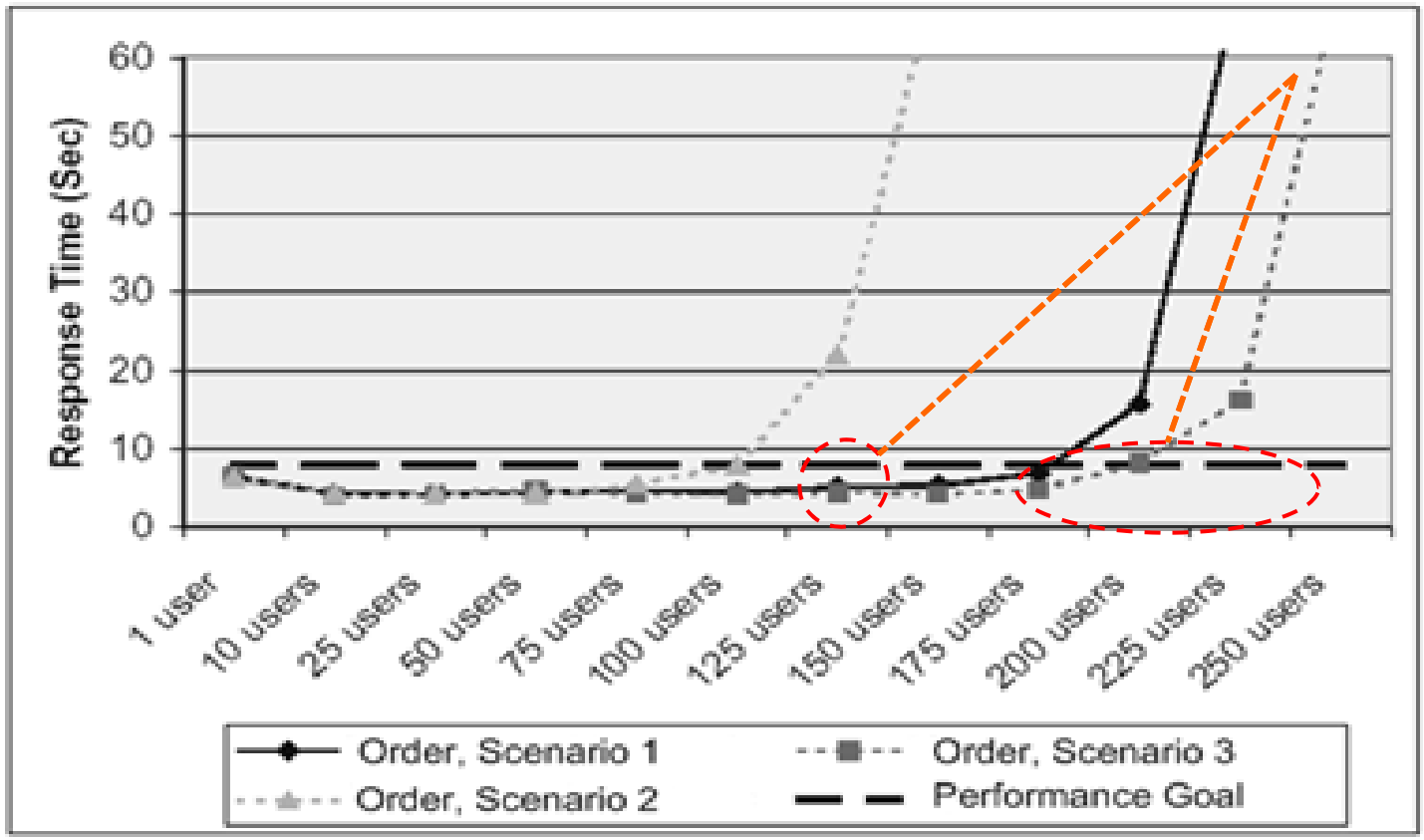
✧ 时间维：尝试观察系统在一段较长时间上的行为变化

✧ 负载维：尝试在系统上改变负载来进行对比分析

✧ 系统维：负载测试也可以针系统的不同组件、不同配置等进行

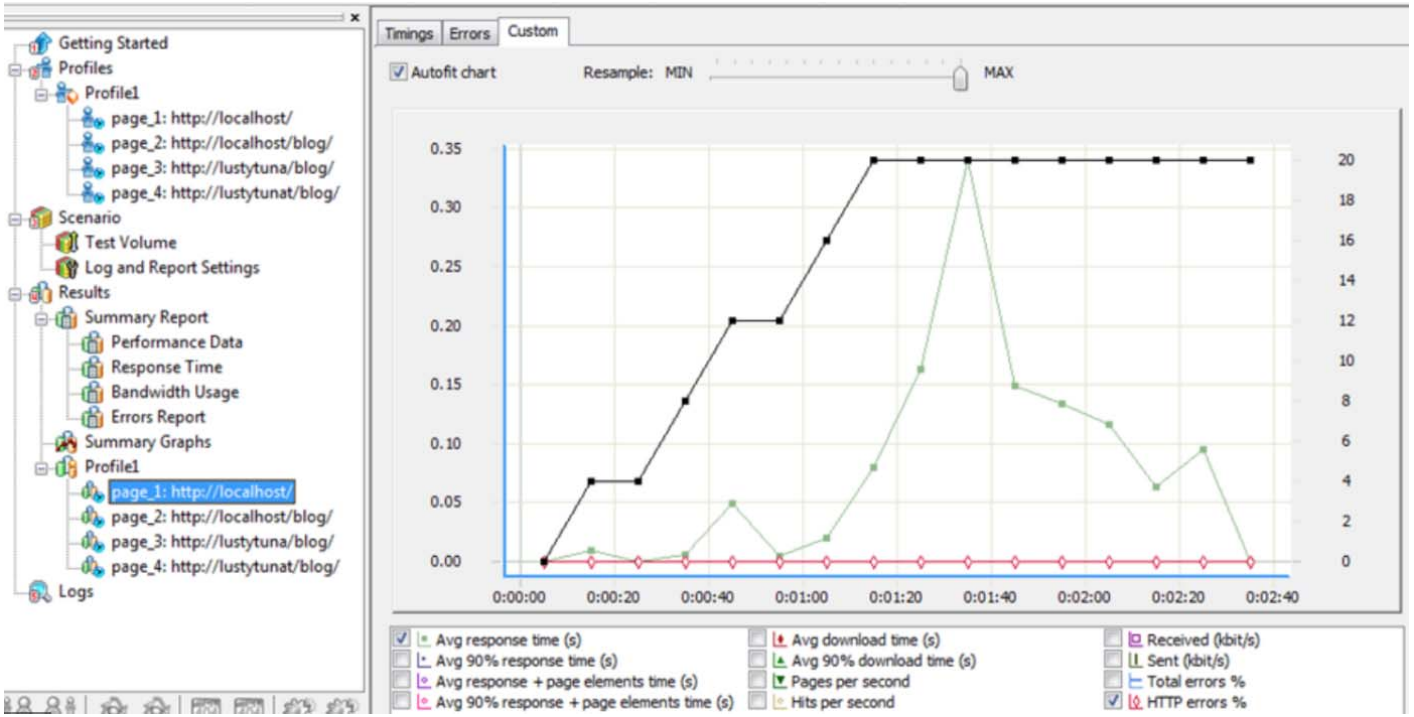
结果分析

要善于捕捉被监控的数据曲线发生突变的地方——拐点



压力测试

压力测试是在系统（如CPU、内存和网络带宽等）处于饱和状态下，测试系统是否还具有正常的会话能力、数据处理能力或是否会出现错误，以检查软件系统对异常情况的抵抗能力，找出性能瓶颈、功能不稳定性等问题。



压力测试的类型

- 稳定性压力测试，高负载下持续运行24小时以上的压力测试
- 破坏性压力测试，通过不断加载的手段，快速造成系统的崩溃，让问题尽快地暴露出来
- 渗入测试（soak test），通过长时间运行，使问题逐渐渗透出来，从而发现内存泄漏、垃圾收集（GC）或系统的其他问题，以检验系统的健壮性
- 峰谷测试（peak-rest test），采用高低突变加载方式进行，先加载到高水平的负载，然后急剧降低负载，稍微平息一段时间，再加载到高水平的负载，重复这样过程，容易发现问题的蛛丝马迹，最终找到问题的根源。

用例编号：LI_001

标题：高并发用户登录响应时间测试

目的：验证200并发用户登录时响应时间≤3秒

预置条件：

- 系统稳定运行，数据库连接正常
- 200个虚拟用户配置完成 2

步骤：

1. 使用JMeter配置200线程组，设置登录请求脚本
2. 执行测试并监控响应时间、TPS、CPU使用率
3. 记录实际数据并与预期对比

预期结果：

- 响应时间≤3秒
- 错误率≤0.1%
- CPU利用率≤75% 2

课堂问题:如何确定性能需求

1、性能测试指标

2、负载的加载方式

响应时间——用户关注点

容量和吞吐量——业务处理方面的关注点

系统资源占用率——开发团队技术关注点

1000个在线用户按正常操作速度访问网上购物系统的下定单功能，下定单交易的成功率是100%，而且90%的下定单请求响应时间不大于5s;当并发在线用户数达到1万个以上时下定单交易的成功率大于 98%，其中 90%的在线用户的请求响应时间不大于用户的最大容忍时间30s。

回顾:常见的性能指标

响应时间

- 指从客户端**发起请求开始**，到客户端**接收到结果**的总时间。
- 包括：**服务器处理时间** + **网络传输时间**

并发数

- 某一时刻**同时**向服务器**发送请求**的用户数

吞吐量

- 指的是**单位时间内**处理的客户端**请求数量**，直接体现软件系统的性能承载能力。**TPS**（每秒事务数）、**QPS**（每秒请求数）

点击数

- 所有的**页面元素**（如：图片、链接、框架等）的**请求总数量**

错误率

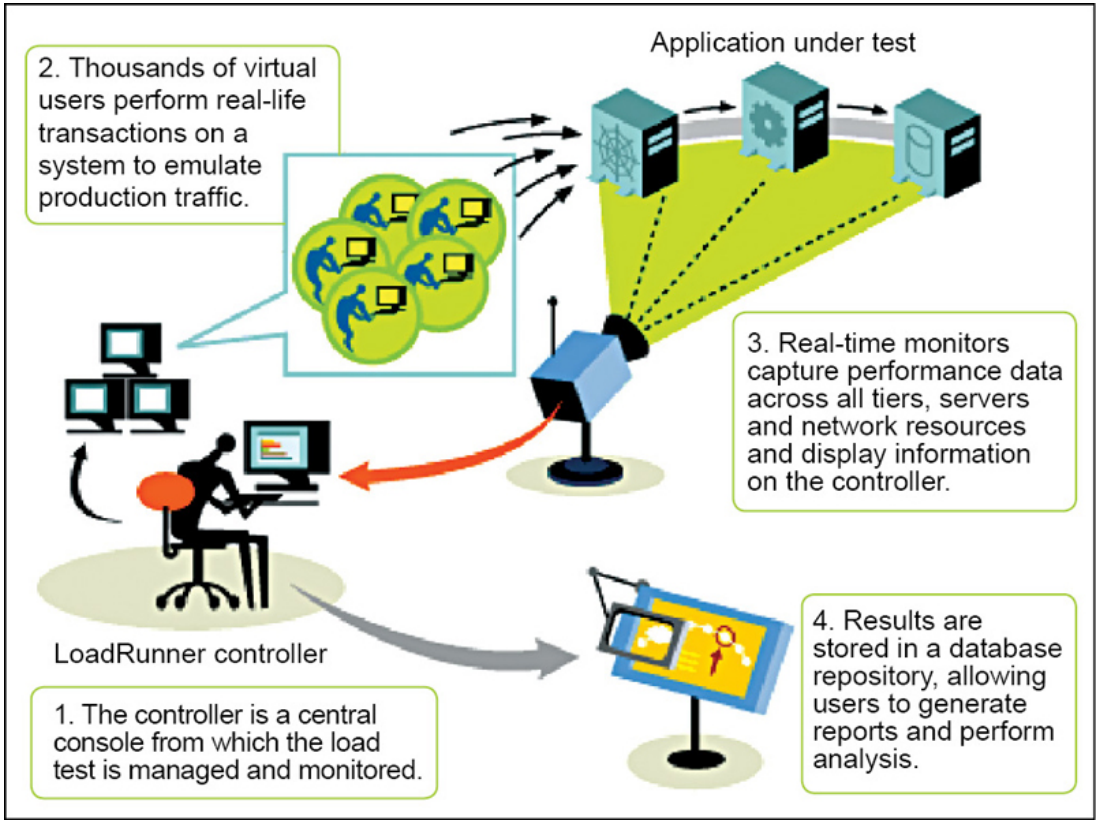
- 指系统在**负载情况**下，失败业务的概率

资源利用率

- 系统各种资源的使用情况，**“资源的使用量/总的资源可用量×100%”**

性能测试工具

- 特性及其使用
- 开源工具
- 商业工具



特性及其使用

- 能模拟实际用户的操作行为，记录和回放多用户测试中的事务处理过程，自动生成相应的测试脚本
- 能针对脚本进行修改，增加逻辑控制、完成参数化和数据关联
- 可以设置不同的应用环境和场景，通过虚拟用户执行相应的测试脚本
- 通过系统监控工具获得系统性能的相关指标的值

补充：主流测试工具

Loadrunner

- HP Loadrunner是一种工业级标准性能测试负载工具，可以模拟上万用户实施测试，并在测试时可实时检测应用服务器及服务器硬件各种数据，来确认和查找存在的瓶颈
- 支持多协议：Web(HTTP/HTML)、Windows Sockets、FTP、ODBC、MS SQL Server等协议
- 采用C语言编写

优点：

1. 多用户（支持用户以万为单位）
2. 详细的分析报表（以秒为单位）
3. 支持IP欺骗功能

缺点：

1. 收费
2. 体积庞大（安装包单位GB）
3. 无法定制功能

传智教育-黑马程序员

补充：主流测试工具

JMeter

- JMeter是Apache组织开发的基于Java的开源软件，用于对系统做功能测试和性能测试。
- 它最初被设计用于Web应用测试，但后来扩展到其他测试领域，例如静态文件、Java 程序、shell 脚本、数据库、FTP、 Mail等。

优点:

1. 开源免费
2. 小巧（安装包50MB左右）
3. 丰富的学习资料和扩展组件

缺点:

1. 不支持IP欺骗
2. 分析和报表能力相对于LR欠缺精度（以分钟为单位）

传智教育-黑马程序员

补充：主流测试工具

小结

（1）主流性能测试工具Loadrunner和Jmeter对比 —— 相同点

- 都能**模拟大量用户**
- 都能**支持多协议**（常见的协议都支持，如：HTTP）
- 都有**监控及分析报表**功能

（2）主流性能测试工具Loadrunner和Jmeter对比 —— 不同点

工具	用户量	分析报表	IP欺骗功能	费用	体积	扩展性
Loadrunner	多（万）	精确（秒）	支持	收费	大（单位GB）	不能扩展
Jmeter	少	较差（分钟）	不支持	免费	小（50MB）	有扩展组件

结论：项目日常性能测试Jmeter足够用，出商业报告优先Loadrunner

开源工具

- **nGrinder**（http://www.nhnopensource.org/ngrinder/）是一个基于Grinder 开发的、易于管理和使用的、分布式性能测试系统。
- **Flood**（http://httpd.apache.org/test/flood/）是人们经常使用的一个**Web**性能测试工具
- **Siege**（http://www.joedog.org/JoeDog/Siege）是一个开源的**Web**压力测试和评测工具。
- **OpenSTA**，可以模拟大量的虚拟用户来完成性能测试，并通过**scrīpt**来完成丰富的自定义设置。详见http://portal.opensta.org/index.php。
- **DBMonster**是一个生成随机数据、用来测试**SQL**数据库的压力测试工具，详见http://dbmonster.kernelpanic.pl/。
- **LoadSim——**网络应用程序的负载模拟器。
- 更多的性能测试工具，可访问 http://www.opensourcetesting.org/performance.php

1. JMeter简介

- 对于互联网应用，可扩展性（Scalability）是一个重要的性能指标。JMeter是Apache组织的开放源代码项目，它是功能和性能测试的工具，100%用Java实现，可以到http://jmeter.apache.org/index.html 下载源代码并查看相关文档。
- JMeter可以用于测试静态或者动态资源的性能（文件、Servlets、Perl脚本、Java对象、数据库和查询、FTP服务器或其他的资源）。JMeter用于模拟在服务器、网络或者其他对象上附加高负载以测试它们提供服务的受压能力，或者分析它们提供的服务在不同负载条件下的总性能情况。可以用JMeter提供的图形化界面分析性能指标或在高负载情况下测试服务器/脚本/对象的行为。

2. Jmeter环境安装

安装JDK

官网下载：
http://www.oracle.com/
提示：下载时注意电脑系统是32位或64位

JAVA_HOME=C:\Program Files\Java\jdk1.8.0_151
在环境变量Path中添加:
%JAVA_HOME%\bin;%JAVA_HOME%\jre\bin;

1、JDK下载 → 2、安装JDK → 3、配置环境变量 → 4、校验

双击安装包进行安装，所有步骤选择默认选项即可

1、打开“cmd”命令行
2、输入 java -version，校验命令能否正常执行以及版本信息是否一致

```
C:\Users\xiaoyang>java -version
java version "1.8.0_191"
Java(TM) SE Runtime Environment (build 1.8.0_191-b12)
Java HotSpot(TM) 64-Bit Server VM (build 25.191-b12, mixed mode)
```

2. Jmeter环境安装

安装JMeter

官网下载地址:
http://jmeter.apache.org/download_jmeter.cgi
提示：Jmeter版本与JDK版本匹配

JMeter_HOME=> F:\Jmeter\apache-jmeter-5.1.1
在环境变量Path中添加:
%JMETER_HOME%\lib\ext\ApacheJMeter_core.jar;%JMETER_HOME%\lib\jorphan.jar



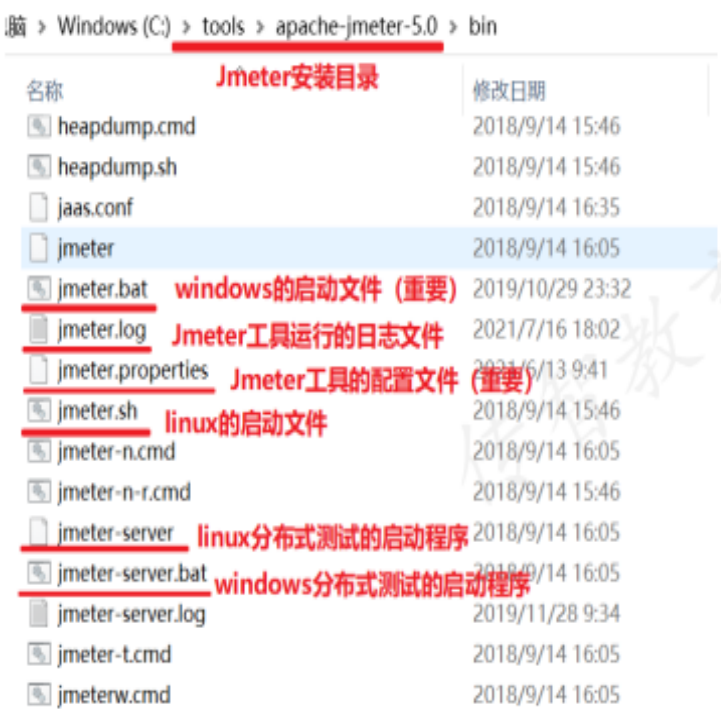
把下载的安装包，解压到某一目录即可
提示：安装目录中不要出现中文/空格

进入JMeter安装目录下的bin目录
1、双击 jmeter.bat
2、双击 ApacheJMeter.jar
3、命令行输入： java -jar ApacheJMeter.jar
注意：任意一种方法能打开即可

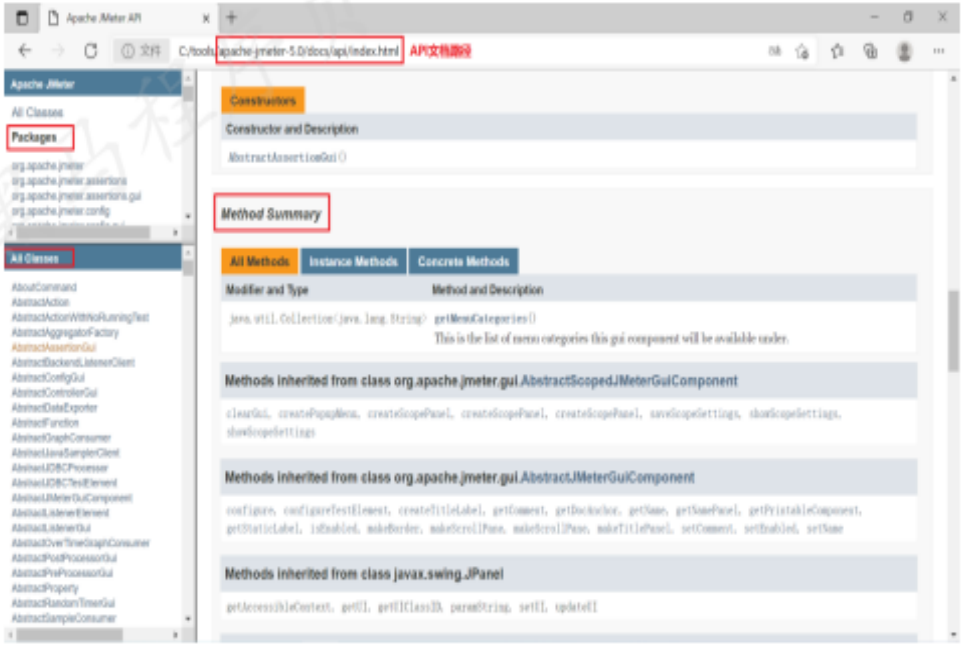
3. Jmeter功能概要

JDK常用文件目录介绍

Bin目录：存放可执行文件和配置文件



docs目录：是JMeter的api文档，用于开发扩展组件



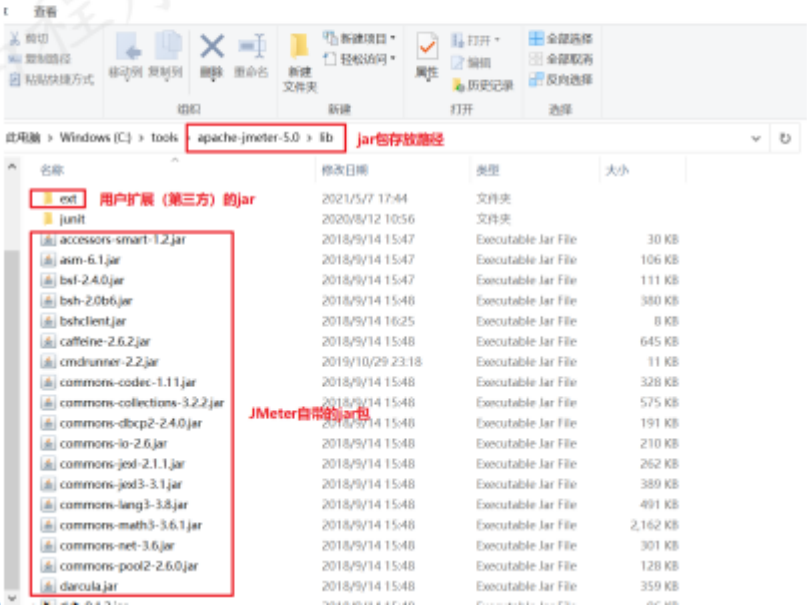
3. Jmeter功能概要

JDK常用文件目录介绍

printable_docs目录：**用户帮助手册**



lib目录：存放JMeter依赖的jar包和用户扩展所依赖的**jar包**

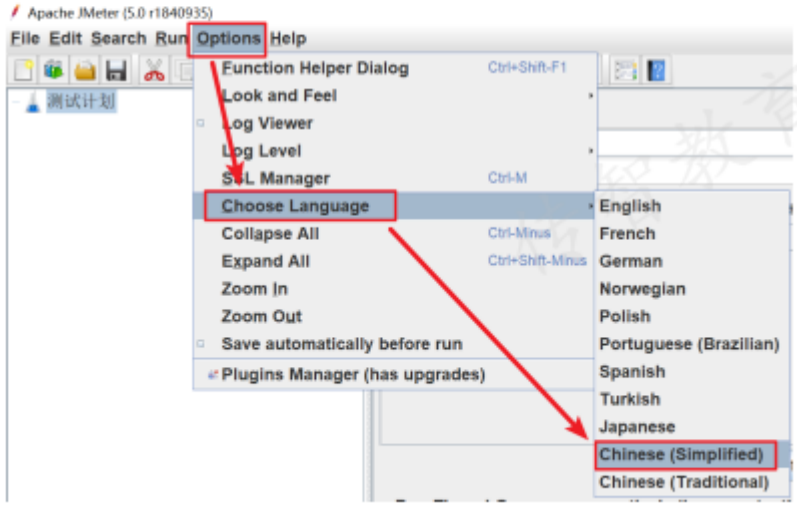


3. Jmeter功能概要

修改默认配置 —— JMeter界面的汉化

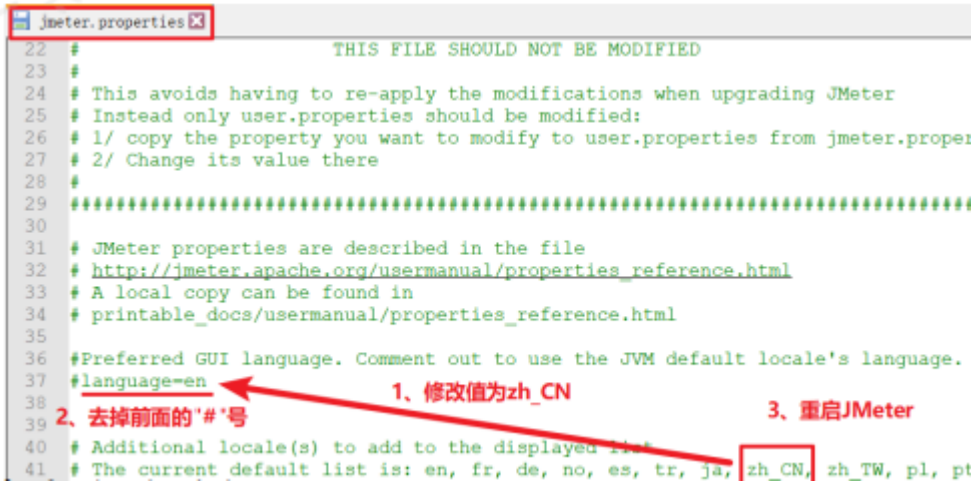
临时性:

启动JMeter-> 选择菜单 ‘Options’ ->Choose Language->Chinese (Simplified)



永久性 —— 修改配置文件:

1. 找到jMeter安装目录下的bin目录
2. 打开jmeter.properties文件，把第37行修改为 “language=zh_CN”
3. 重启JMeter即可



3. Jmeter功能概要



（1）下面几个场景下，需要去哪个目录下获取信息?

- 修改JMeter配置文件 —— **Bin目录**
- 下载第三方插件（jar包）并使用 —— **lib/ext目录**
- 查找用户帮助手册 —— **printable_docs目录**
- 启动Jmeter程序 —— **Bin目录**

（2）如何修改默认配置？

Jmeter界面汉化:

- 永久性：**修改配置文件**， language=zh_CN，重启Jmeter
- 临时性：菜单 ‘Options’ ->Choose Language->Chinese (Simplified)

Jmeter主题修改:

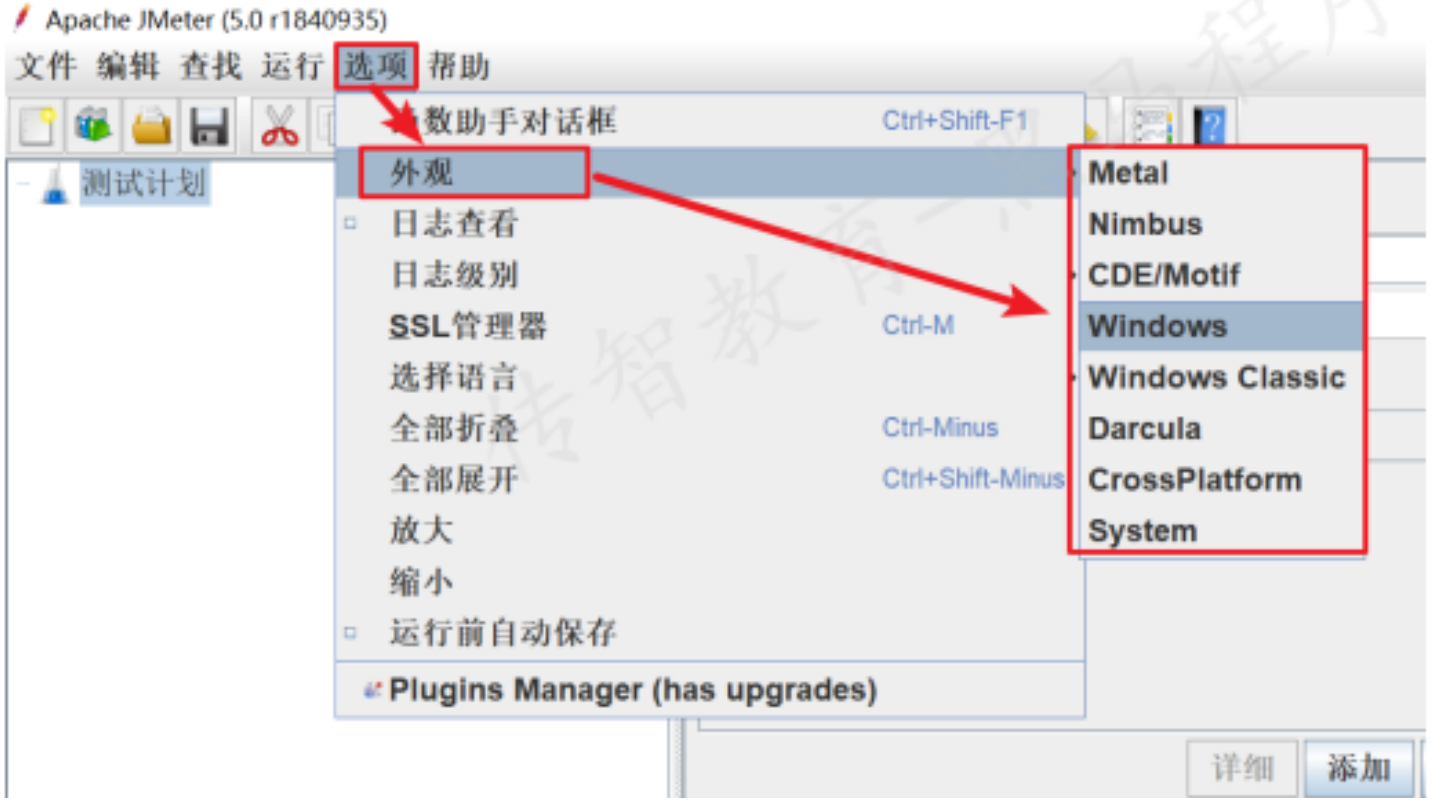
- 菜单 ‘选项’ -> 外观 -> Windows(选择自己喜欢的主题即可)

3. Jmeter功能概要

修改默认配置 —— 修改主题

JMeter默认主题是黑色的，可以通过以下步骤修改：

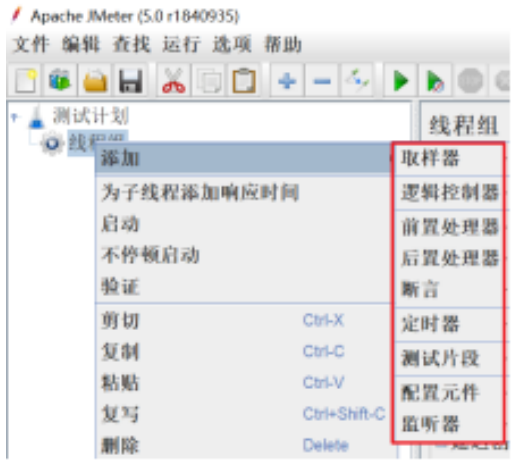
- 启动JMeter -> 选择菜单 ‘选项’ -> 外观 -> Windows(选择自己喜欢的主题即可)



4. Jmeter元件和组件的介绍

元件的基本介绍

元件：多个类似功能组件的**容器**（类似于**类**）



取样器	发送请求
逻辑控制器	控制语句的执行顺序
前置处理器	对请求参数进行预处理
后置处理器	对响应结果进行提取
断言	检查接口的返回结果是否与预期结果一致
定时器	设置等待
测试片段	封装一段代码，供其他脚本调用
配置元件	测试数据的初始化配置
监听器	查看Jmeter脚本的运行结果

4. Jmeter元件和组件的介绍

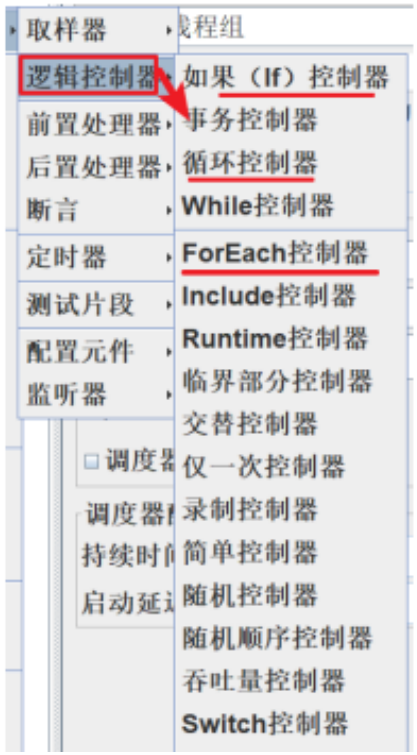
组件的基本介绍

组件：实现独立的某个功能（类似于**方法**）

例如：取样器的组件



例如：逻辑控制器的组件



4. Jmeter元件和组件的介绍



（1）如下接口自动化脚本的实现过程对应着Jmeter哪个元件?

1. 初始化测试数据 —— **配置元件**
2. 对请求参数进行赋值 —— **前置处理器**
3. 调用GET/POST方法发送请求 —— **取样器**
4. 提取响应中特定字段的值 —— **后置处理器**
5. 对提取出来的值与预期结果进行对比 —— **断言**
6. 在控制台查看脚本运行的结果 —— **监听器**

（2）元件与组件有什么关系？

元件：多个类似功能组件的**容器**（类似于**类**）

组件：**容器中**实现独立的某个功能（类似于**方法**）

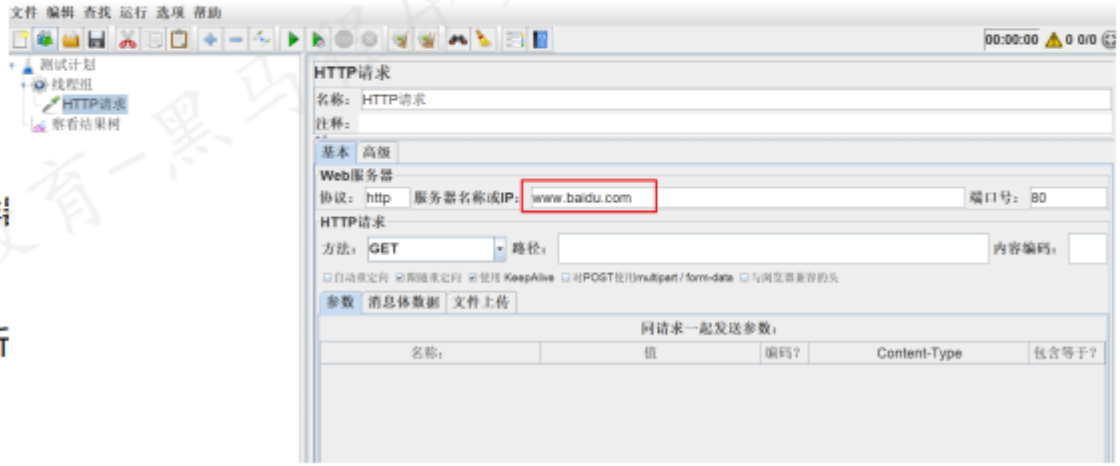
5. Jmeter使用示例

JMeter第一个案例

需求：使用JMeter访问百度首页接口，并查看请求和响应信息

步骤：

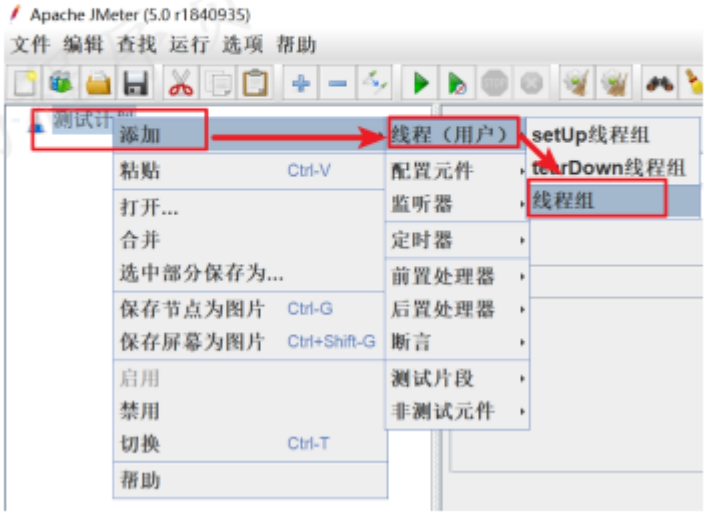
1. 启动JMeter
2. 在‘测试计划’下添加‘**线程组**’
3. 在‘线程组’下添加‘**HTTP请求**’取样
4. 填写‘HTTP请求’的相关请求数据
5. 在‘线程组’下添加‘**察看结果树**’监听
6. 点击‘启动’按钮运行，并查看结果



5. Jmeter使用示例-1：线程组

作用：线程组就是控制JMeter用于执行测试的一组用户

位置：右键点击 ‘测试计划’ --> 添加 --> 线程（用户） --> 线程组



特点：

- 模拟多人操作
- 线程组可以添加多个，多个线程组可以并行或串行
- 取样器（请求）和逻辑控制器必须依赖线程组才能使用
- 线程组下可以添加其他元件下组件

5. Jmeter使用示例-1：线程组

线程组分类：

- **线程组：**

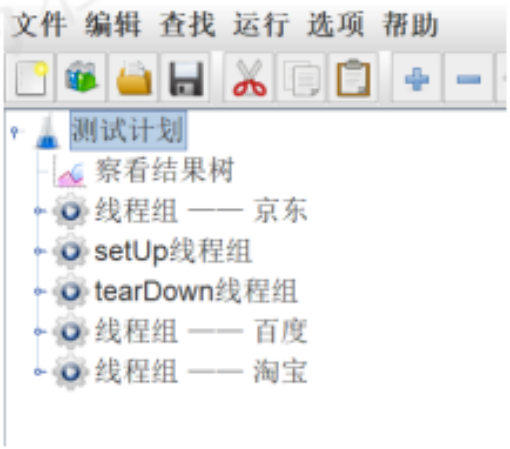
  普通的、常用的线程组，可以看做一个虚拟用户组，线程组中的每一个线程都可以理解为一个虚拟用户

- **setUp线程组：**

  一种特殊类型的线程组，可用于执行预测试操作

- **tearDown线程组：**

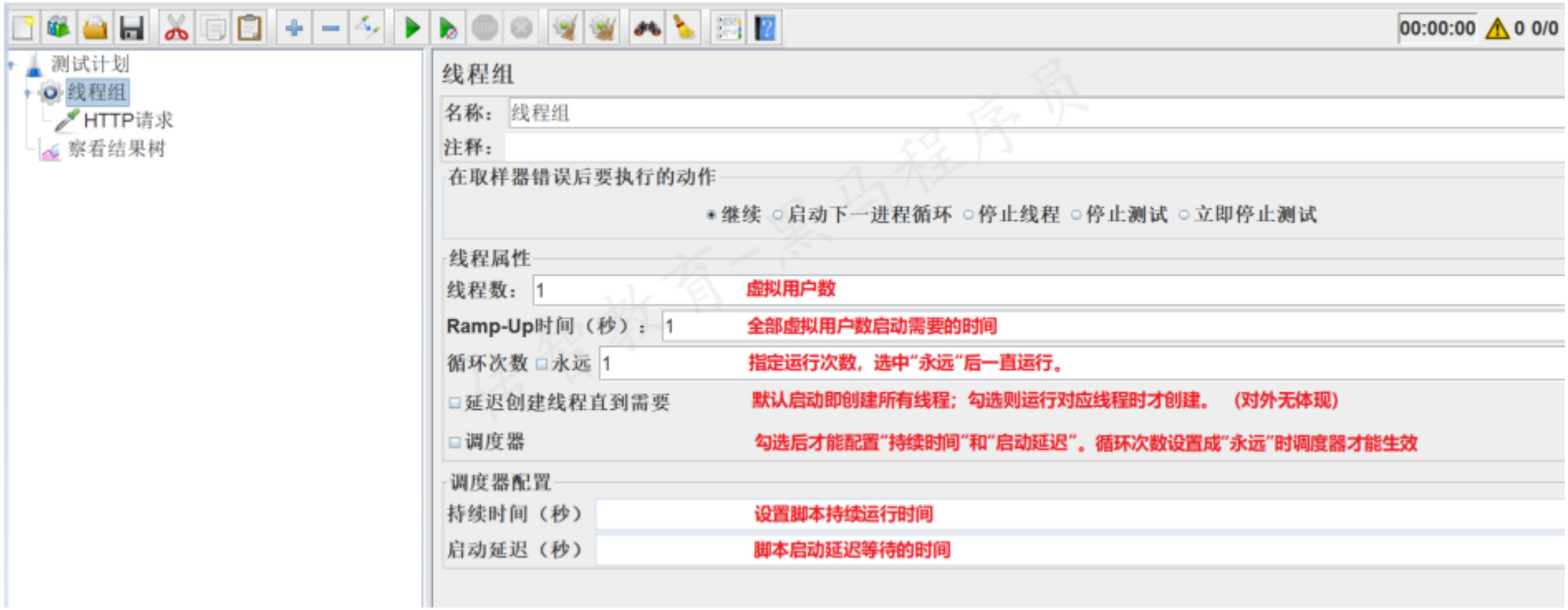
  一种特殊类型的线程组，可用于执行测试后工作



传智教育-黑马程序员

5. Jmeter使用示例-1：线程组

线程组 —— 参数详解



5. Jmeter使用示例-1：线程组



编写JMeter脚本，访问百度

要求：

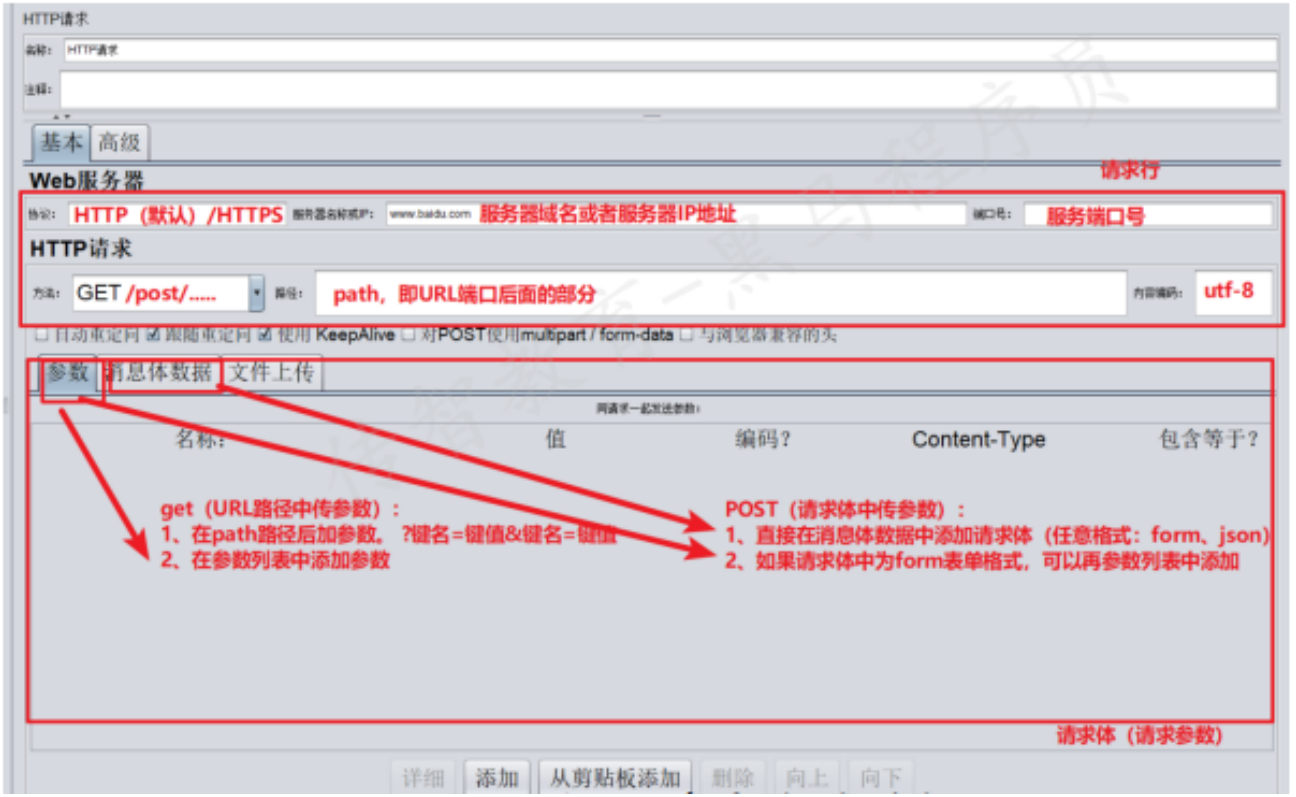
- 模拟10个用户并行执行：
- 模拟10个用户5s内启动完成：
- 模拟2个用户各循环3次：
- 模拟2个用户运行30s：
- 模拟2个用户等待10s后开始执行

5. Jmeter使用示例-2：HTTP请求

作用： 向服务器发送http及https请求

位置： 选中线程组->右键->添加->取样器->HTTP请求

参数：



5. Jmeter使用示例-2：HTTP请求

练习　编写Jmeter脚本，访问百度

案例一：GET请求，URL为http://www.baidu.com/S?wd=test

要求：使用HTTP请求 - 路径来传递get请求参数

案例二：GET请求，URL为https://www.baidu.com/S?wd=test

要求：使用HTTP请求 -参数列表来传递get请求的参数

案例三：POST请求，URL为https://www.baidu.com/S，请求体为:wd=test(form表单)

要求：使用HTTP请求 - 参数列表来传递POST请求的form格式参数

案例四：POST请求，URL为http://www.baidu.com/S，请求体为:wd=test(form表单)

要求：使用HTTP请求 – 消息体数据来传递POST请求的form格式参数

5. Jmeter使用示例-2：HTTP请求



（1）如何配置来发送HTTP请求?

发送GET请求（请求参数在路径中）：

- **URL**：协议、服务器域名或IP、端口、方法(GET)、路径、编码格式
- **参数:**
  - 在路径后添加参数。格式：?键名=键值&键名=键值
  - 在参数列表中添加参数。格式：名称（键名）、值（键值）

发送POST请求（请求参数在消息体数据中）：

- **URL**：协议、服务器域名或IP、端口、方法(POST)、路径、编码格式
- **参数:**
  - 在消息体数据中添加请求体（form/json）
  - 在参数列表中添加参数（form）

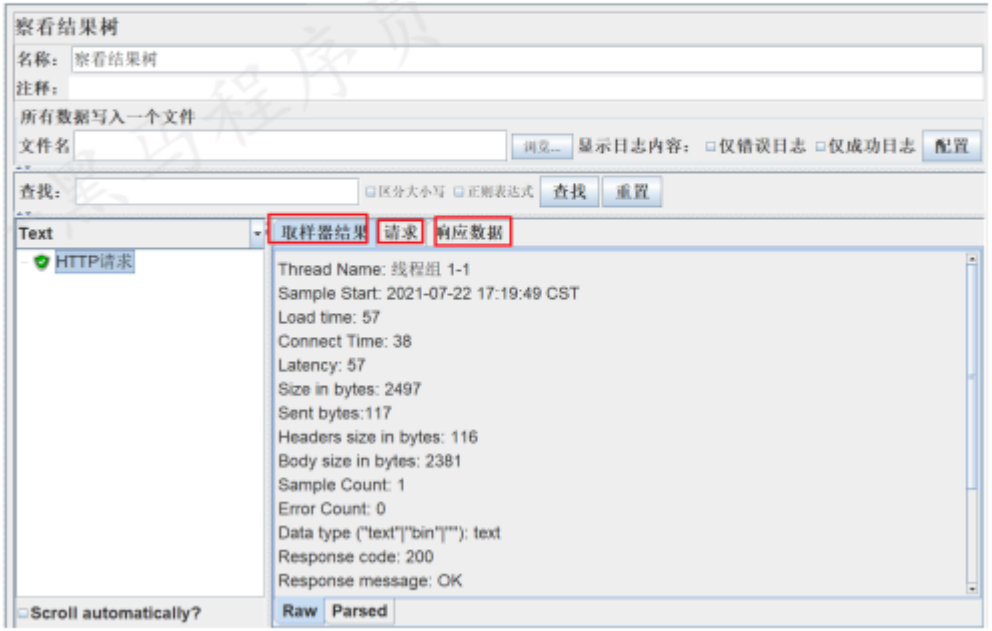
5. Jmeter使用示例-3：查看结果树

作用：查看HTTP请求的请求和响应结果

位置：选中测试计划/线程组->右键->添加->监听器->察看结果树

组成：

- 取样结果：查看响应信息头信息、响应状态码
- 请求：查看请求相关信息（url、方法、参数）
- 响应：查看响应信息



（1）如何查看HTTP消息请求和响应内容?

如何查看请求参数：

- 请求 – Request Body （请求行 + 请求体）

如何查看响应结果：

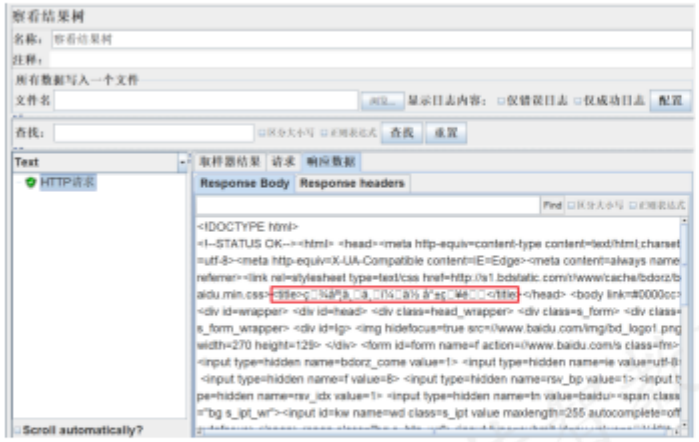
- 响应数据 – Response Body（响应体）

5. Jmeter使用示例-3：查看结果树

（2）如何解决Jmeter查看结果树的响应中的中文乱码？

修改配置文件

结果出现乱码：



解决方法：

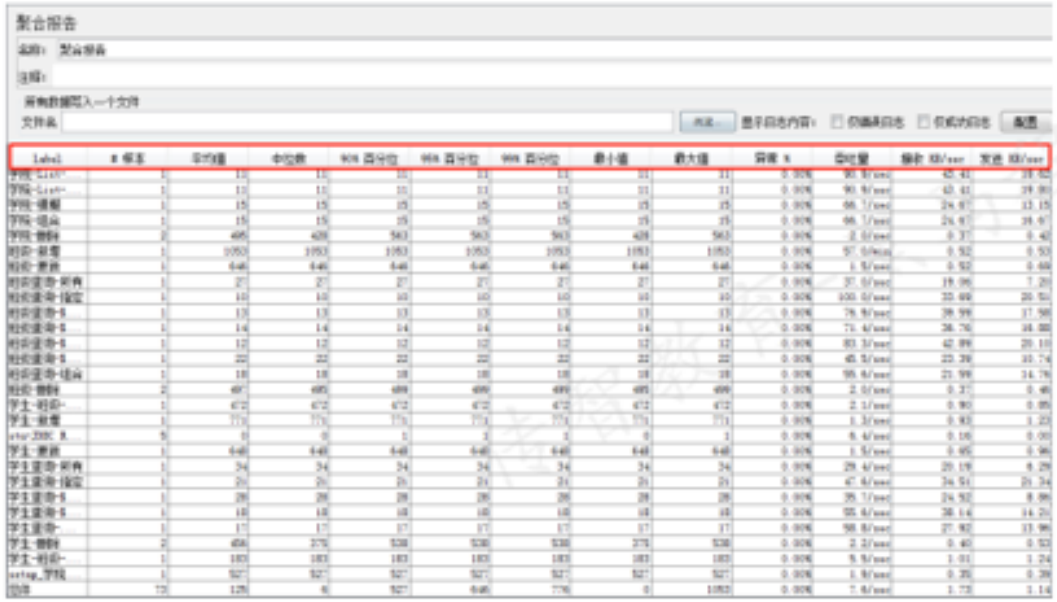
1. 找到jMeter安装目录下的bin目录
2. 打开jmeter.properties文件，修改配置 sampleresult.default.encoding=UTF-8
3. 重启JMeter即可

```
1061  # The encoding to be used if none is provided (default ISO-8859-1)
1062  #sampleresult.default.encoding=ISO-8859-1   1、改为UTF-8        3、重启jmeter
1063        2、去掉#
1064  # CookieManager behaviour - should cookies with null/empty values be deleted?
1065  # Default is true. Use false to revert to original behaviour
1066  #CookieManager.delete_null_cookies=true
```

5. Jmeter使用示例-4：聚合报告

作用：收集性能测试结束后，系统的各项性能指标。如：响应时间、并发数、吞吐量、错误率等

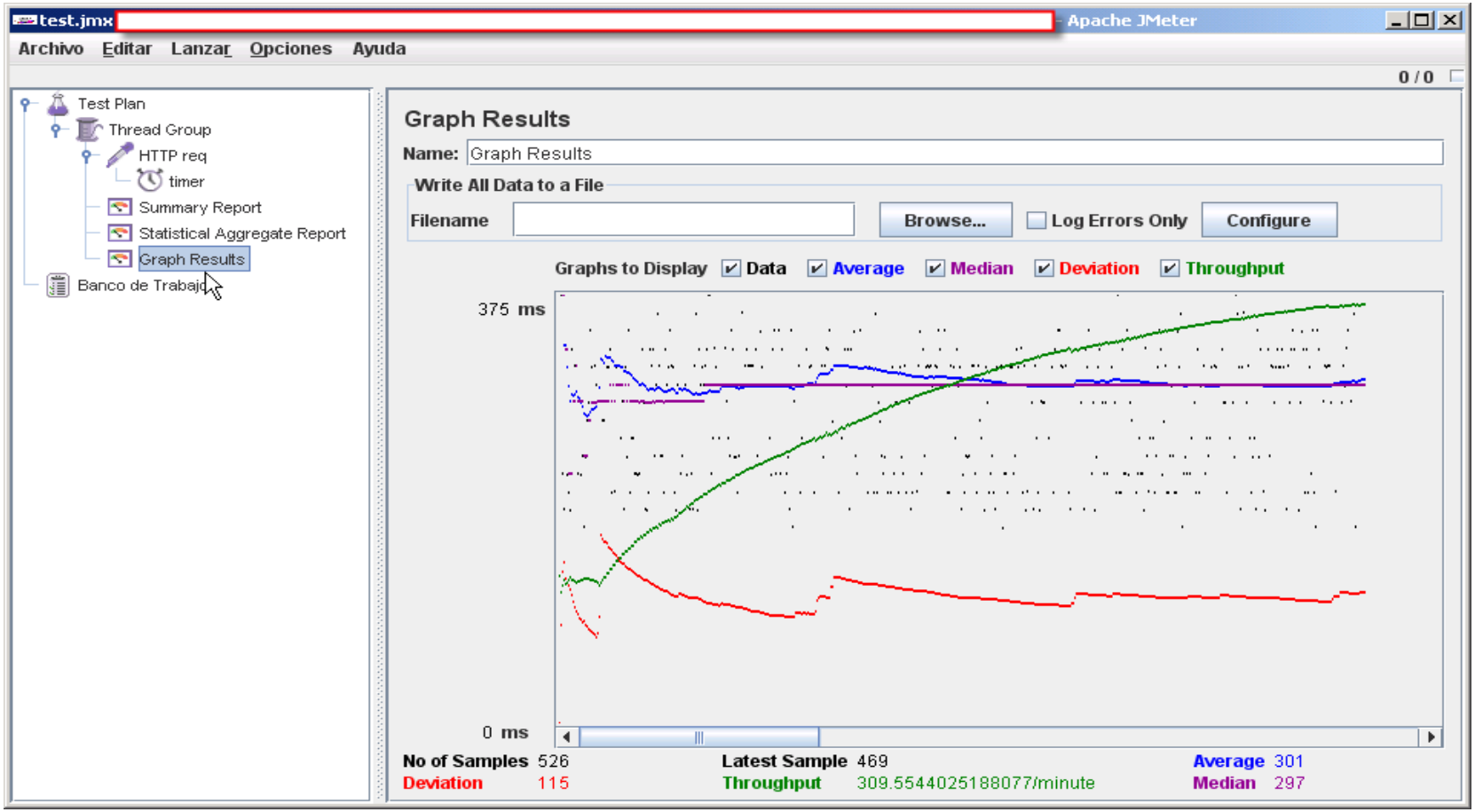
位置：测试计划->右键->监听器->聚合报告



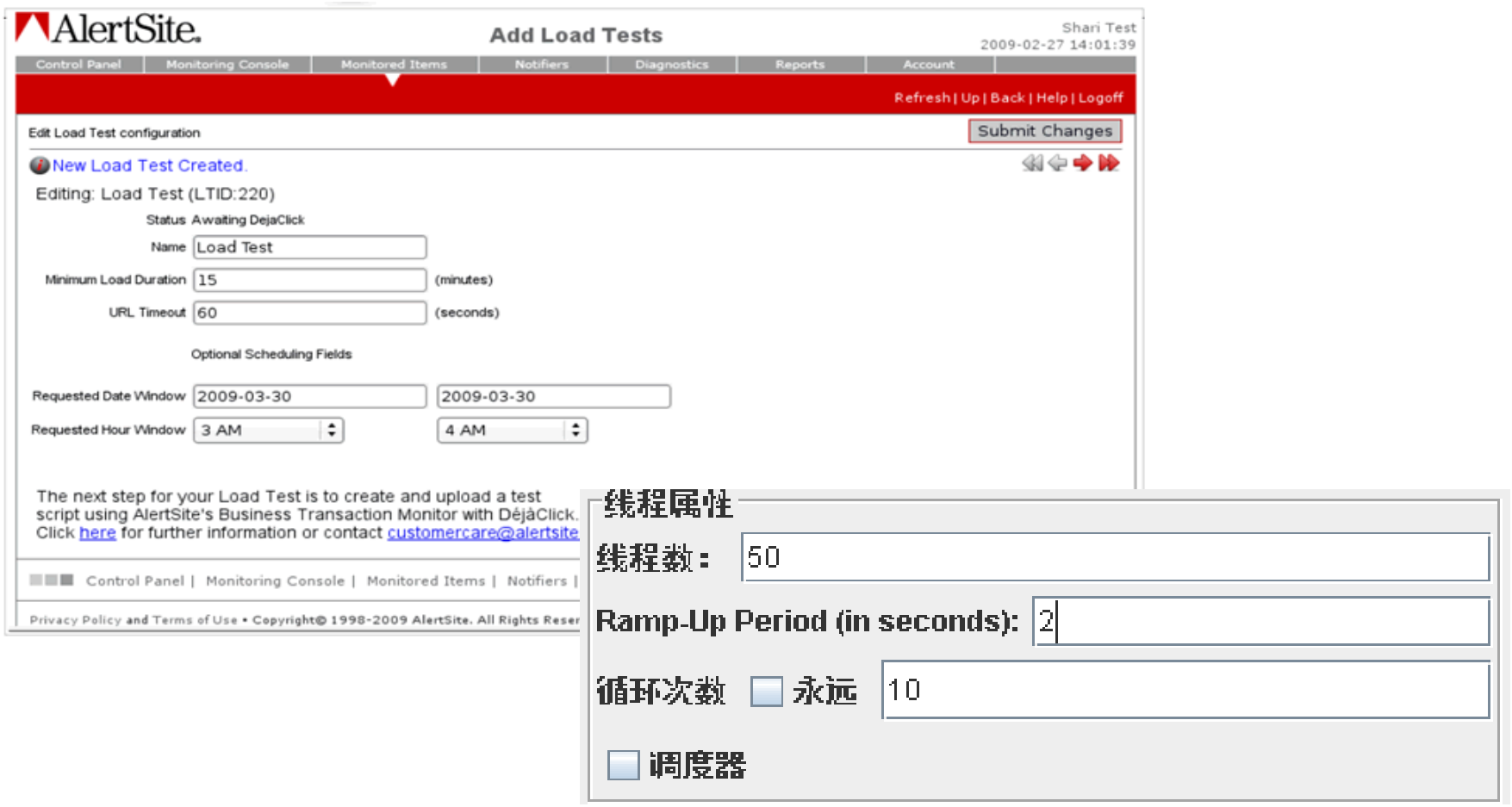
参数介绍：

- **Label：**每个请求的名称
- **样本：**各请求发出的数量
- **平均值：**平均响应时间（单位：毫秒）
- **中位数：**中位数，50% <= 时间
- **90%百分比：**90% <= 时间
- **95%百分比：**95% <= 时间
- **99%百分比：**99% <= 时间
- **最小值：**最小响应时间
- **最大值：**最大响应时间
- **异常%：**请求的错误率
- **吞吐量：**吞吐量。默认情况下表示每秒完成的请求数，一般认为它为TPS。
- **接收 KB/sec：**每秒接收到的千字节数
- **发送 KB/sec：**每秒发送的千字节数

5. Jmeter使用示例-5：图形结果

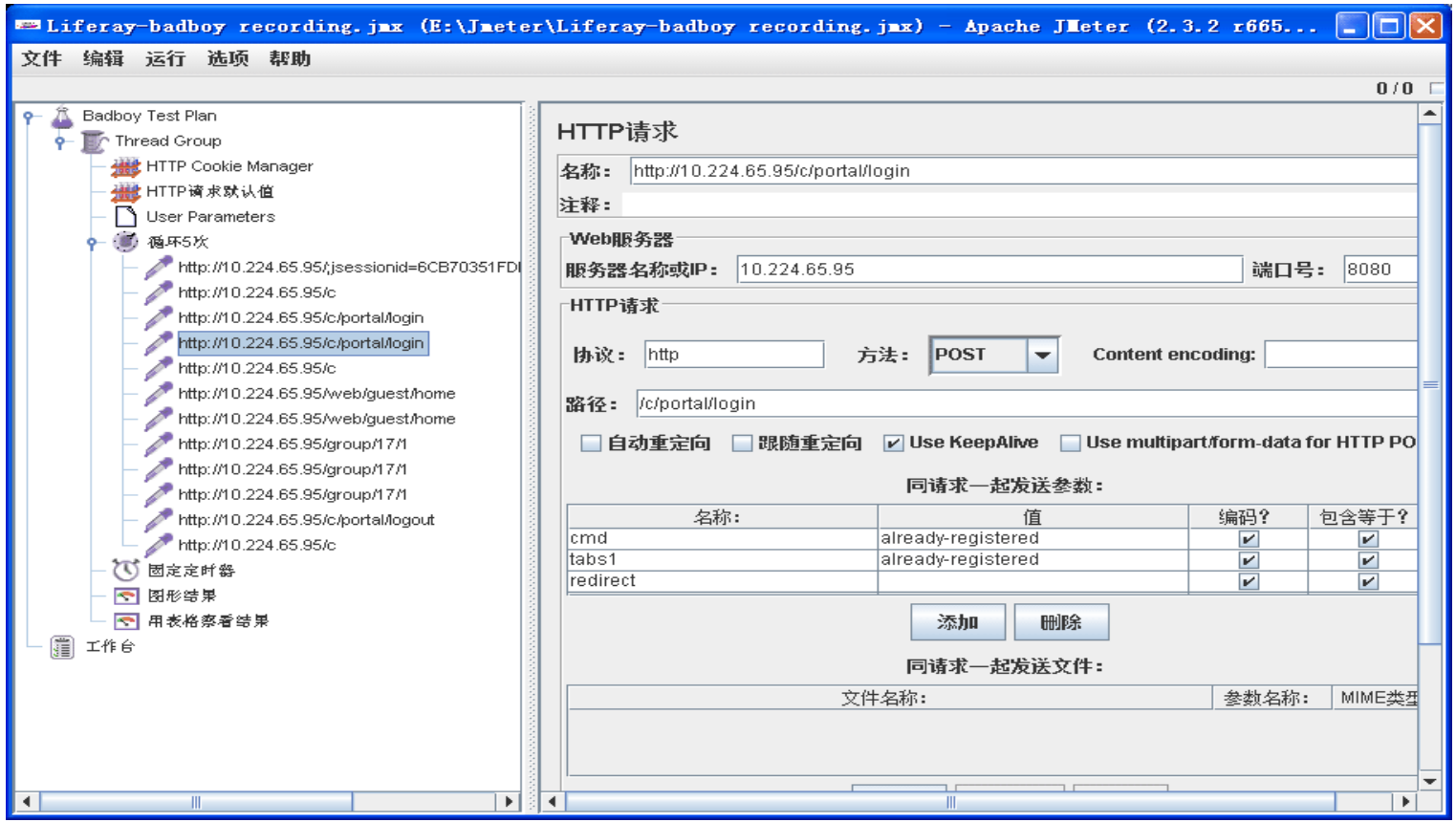


参数实例


AlertSite.
Add Load Tests
Shari Test
2009-02-27 14:01:39
Control Panel
Monitoring Console
Monitored Items
Notifiers
Diagnostics
Reports
Account
Refresh | Up | Back | Help | Logoff
Edit Load Test configuration
Submit Changes
New Load Test Created.
Editing: Load Test (LTID:220)
Status Awaiting DejaClick
Name Load Test
Minimum Load Duration 15 (minutes)
URL Timeout 60 (seconds)
Optional Scheduling Fields
Requested Date Window 2009-03-30 2009-03-30
Requested Hour Window 3 AM 4 AM
The next step for your Load Test is to create and upload a test script using AlertSite's Business Transaction Monitor with DéjàClick. Click here for further information or contact customercare@alertsite
Control Panel | Monitoring Console | Monitored Items | Notifiers |
Privacy Policy and Terms of Use • Copyright© 1998-2009 AlertSite. All Rights Reser
线程属性
线程数： 50
Ramp-Up Period (in seconds): 2
循环次数 永远 10
调度器

Demo

JMeter


Liferay-badboy recording.jmx (E:\Jmeter\Liferay-badboy recording.jmx) - Apache JMeter (2.3.2 r665...
文件 编辑 运行 选项 帮助
0 / 0
Badboy Test Plan
Thread Group
HTTP Cookie Manager
HTTP请求默认值
User Parameters
循环5次
http://10.224.65.95/;jsessionid=6CB70351FDI
http://10.224.65.95/c
http://10.224.65.95/c/portal/login
http://10.224.65.95/c/portal/login
http://10.224.65.95/c
http://10.224.65.95/web/guest/home
http://10.224.65.95/web/guest/home
http://10.224.65.95/group/17/1
http://10.224.65.95/group/17/1
http://10.224.65.95/group/17/1
http://10.224.65.95/c/portal/logout
http://10.224.65.95/c
固定定时器
图形结果
用表格察看结果
工作台
HTTP请求
名称: http://10.224.65.95/c/portal/login
注释:
Web服务器
服务器名称或IP: 10.224.65.95
端口号: 8080
HTTP请求
协议: http
方法: POST
Content encoding:
路径: /c/portal/login
自动重定向
跟随重定向
Use KeepAlive
Use multipart/form-data for HTTP PO
同请求一起发送参数:
名称:
值
编码?
包含等于?
cmd
already-registered
tabs1
already-registered
redirect
添加
删除
同请求一起发送文件:
文件名称:
参数名称:
MIME类型

输出参数

- 数据传输的吞吐量（Transactions）
- 数据处理效率（Transactions per second）
- 数据请求的响应时间（Response time）
- 内存和CPU使用率
- 连接时间（Connect Time）、发送时间（Sent Time）
- 处理时间（Process Time）、页面下载时间
- 第一次缓冲时间
- 每秒（SSL）连接数
- 每秒事务总数、每秒下载页面数
- 每秒点击次数、每秒HTTP 响应数
- 每秒重试次数

举例

聚合报告

名称： 聚合报告

注释：

所有数据写入一个文件

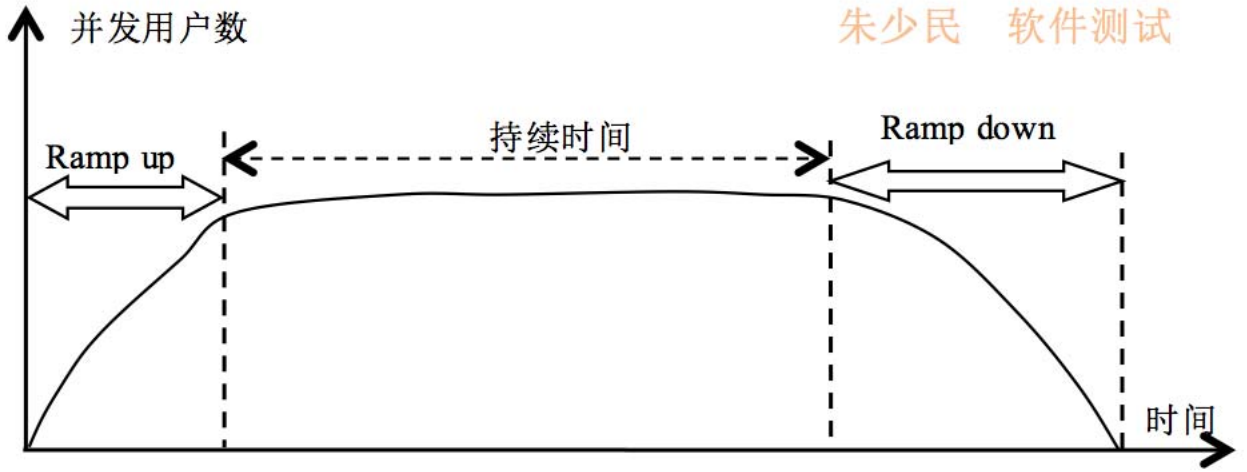
文件名 浏览... Log/Display Only: 仅日志错误 Successes Configure

Label	# Samples	Average	Median	90% Line	Min	Max	Error %	Throughput	KB/sec
http://localh...	150	253	198	487	3	533	0.00%	83.8/sec	16.2
http://localh...	450	6182	8737	11361	33	15104	0.00%	16.1/sec	101.7
http://localh...	300	194	58	275	3	2532	0.00%	4.3/sec	.0
http://localh...	750	9310	8475	16566	968	23804	0.00%	10.9/sec	223.1
http://localh...	150	163	131	251	4	1060	0.00%	6.9/sec	.4
总体	1800	5492	4555	11919	3	23804	0.00%	25.8/sec	262.3

图8-7 JMeter性能测试结果示意图

场景设置

在性能测试执行前，以什么样方式启动负载方式、如何持续进行负载测试直至负载测试结束，这个过程的负载大小和方式、负载启动和结束以及各种检查点、验证点等设计，被称为场景设置。

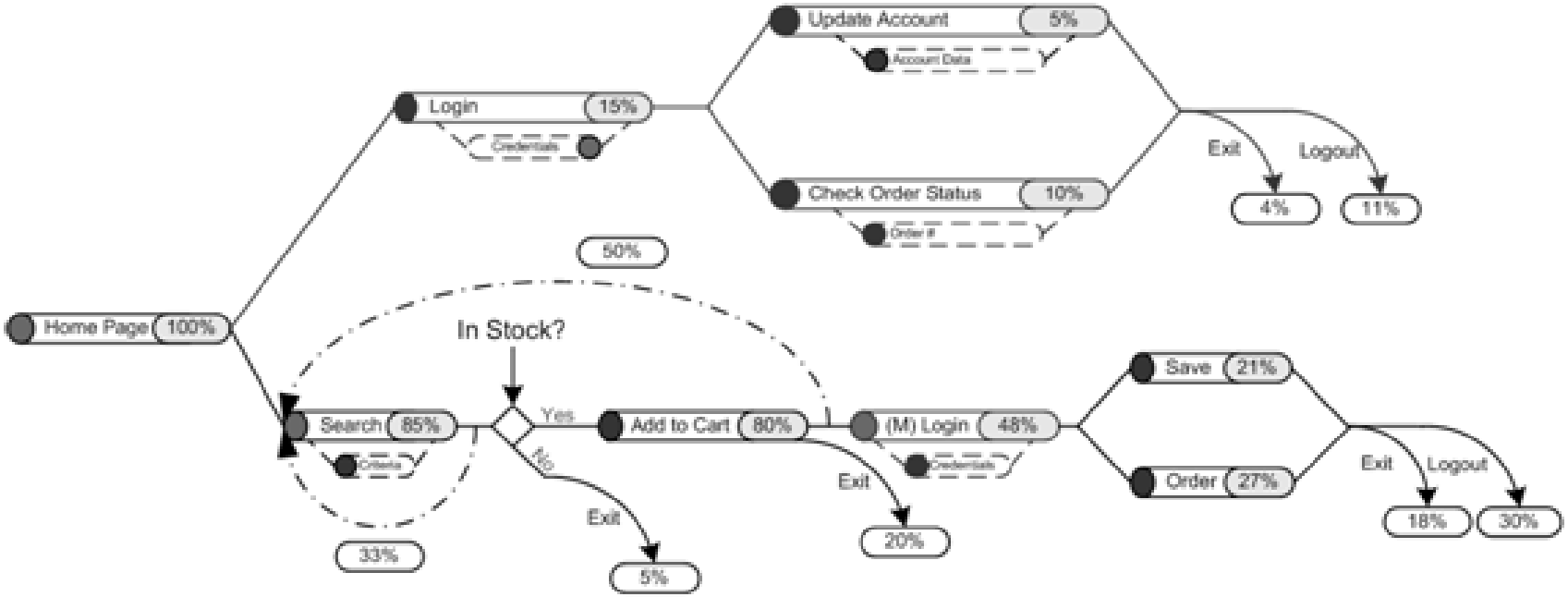


启动（Ramp up） 持续期间（Duration） 结束（Ramp down）

场景类型

- 可以分为静态和动态两部分。静态部分是指设置模拟用户生成器、用户数量、用户组等，动态部分主要指添加性能计数器、检查点、阀值等，从而获得负载测试过程中反回来馈的数据——系统运行的动态状态。
- 可以依据业务模式变化、随时间段变化来进行设置
- 也可分为手工场景 和面向目标的场景

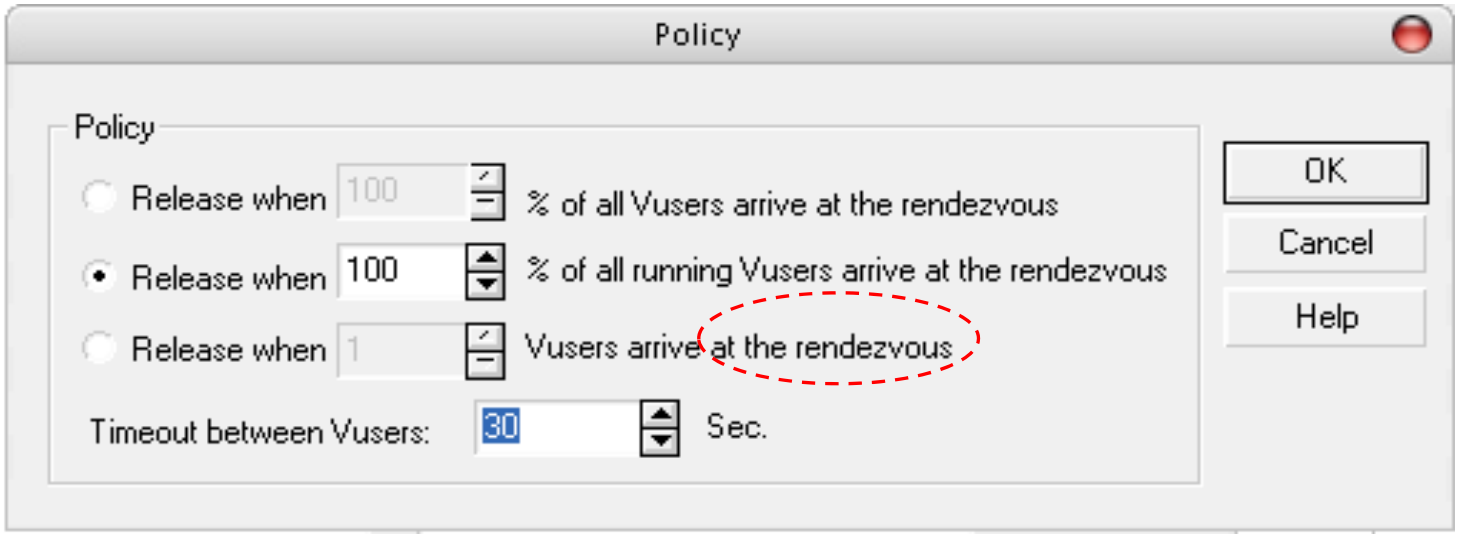
举例



依据业务模式变化来设计负载模式（来源MSDN）

同步点

同步点（或称集合点）用于同步虚拟用户恰好在某一时刻执行任务，确保众多的虚拟并发用户更准确、集中地进行某个设定的操作，达到更理想的负载模拟效果



负载测试执行

- 大量的虚拟用户要运行在多个客户端，并由控制器管理、代理（agent）驱动
- 负载测试的执行，需要针对不同维度的变化进行，包括时间维、负载维和系统维
- 监控、详细的记录和适当的分析是十分重要的

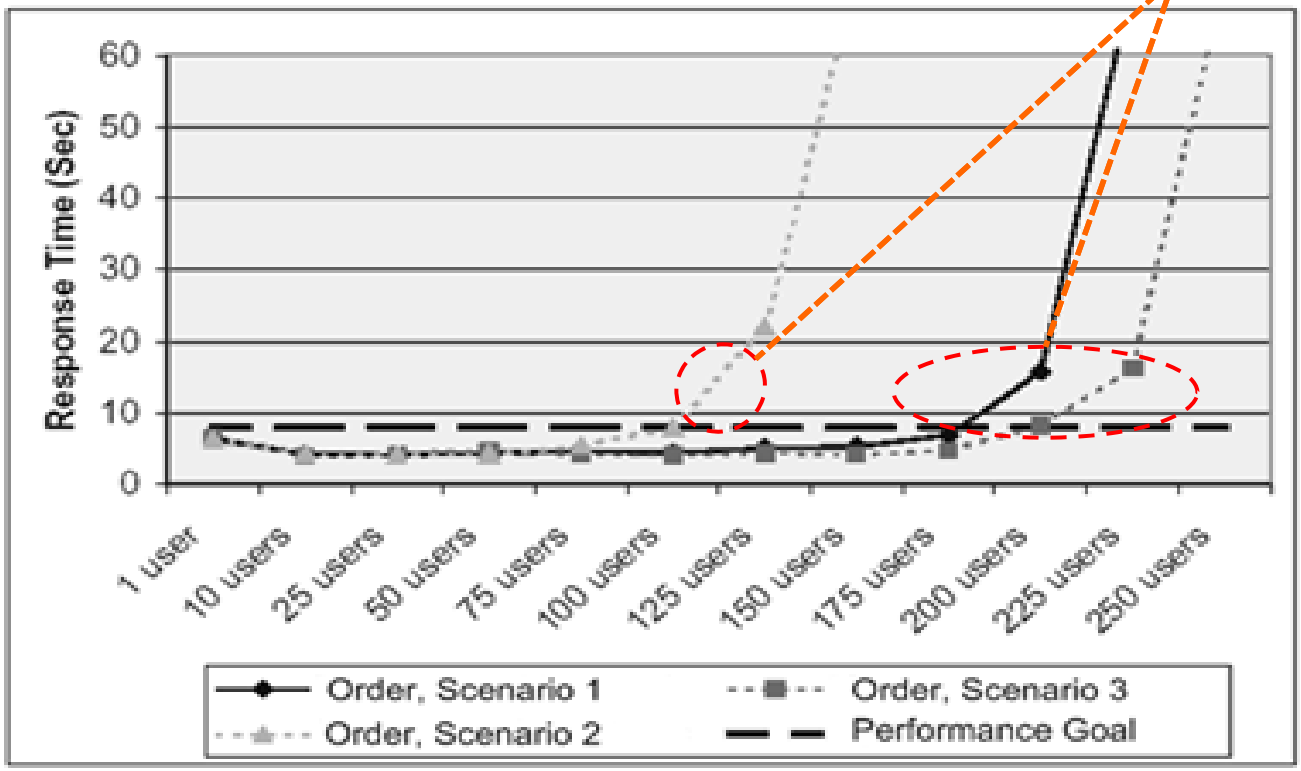
✧ 时间维：尝试观察系统在一段较长时间上的行为变化

✧ 负载维：尝试在系统上改变负载来进行对比分析

✧ 系统维：负载测试也可以针系统的不同组件、不同配置等进行

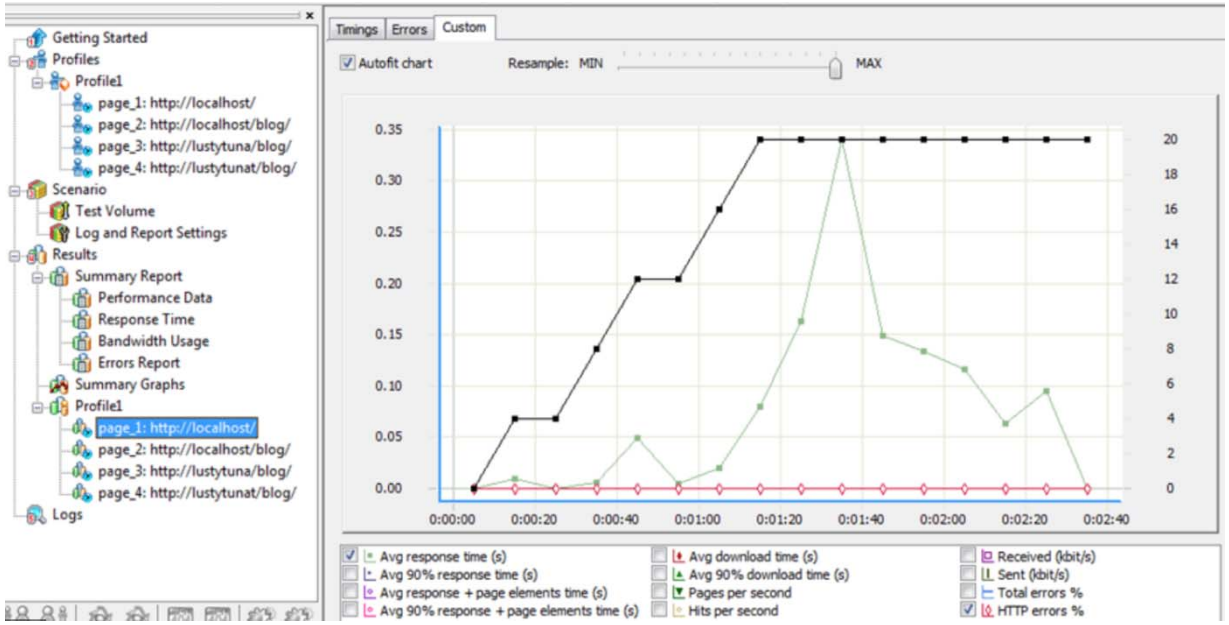
结果分析

要善于捕捉被监控的数据曲线发生突变的地方——拐点



压力测试

压力测试是在系统（如CPU、内存和网络带宽等）处于饱和状态下，测试系统是否还具有正常的会话能力、数据处理能力或是否会出现错误，以检查软件系统对异常情况的抵抗能力，找出性能瓶颈、功能不稳定性等问题。



压力测试的类型

- 稳定性压力测试，高负载下持续运行24小时以上的压力测试
- 破坏性压力测试，通过不断加载的手段，快速造成系统的崩溃，让问题尽快地暴露出来
- 渗入测试（soak test），通过长时间运行，使问题逐渐渗透出来，从而发现内存泄漏、垃圾收集（GC）或系统的其他问题，以检验系统的健壮性
- 峰谷测试（peak-rest test），采用高低突变加载方式进行，先加载到高水平的负载，然后急剧降低负载，稍微平息一段时间，再加载到高水平的负载，重复这样过程，容易发现问题的蛛丝马迹，最终找到问题的根源。

1、测试系统配置

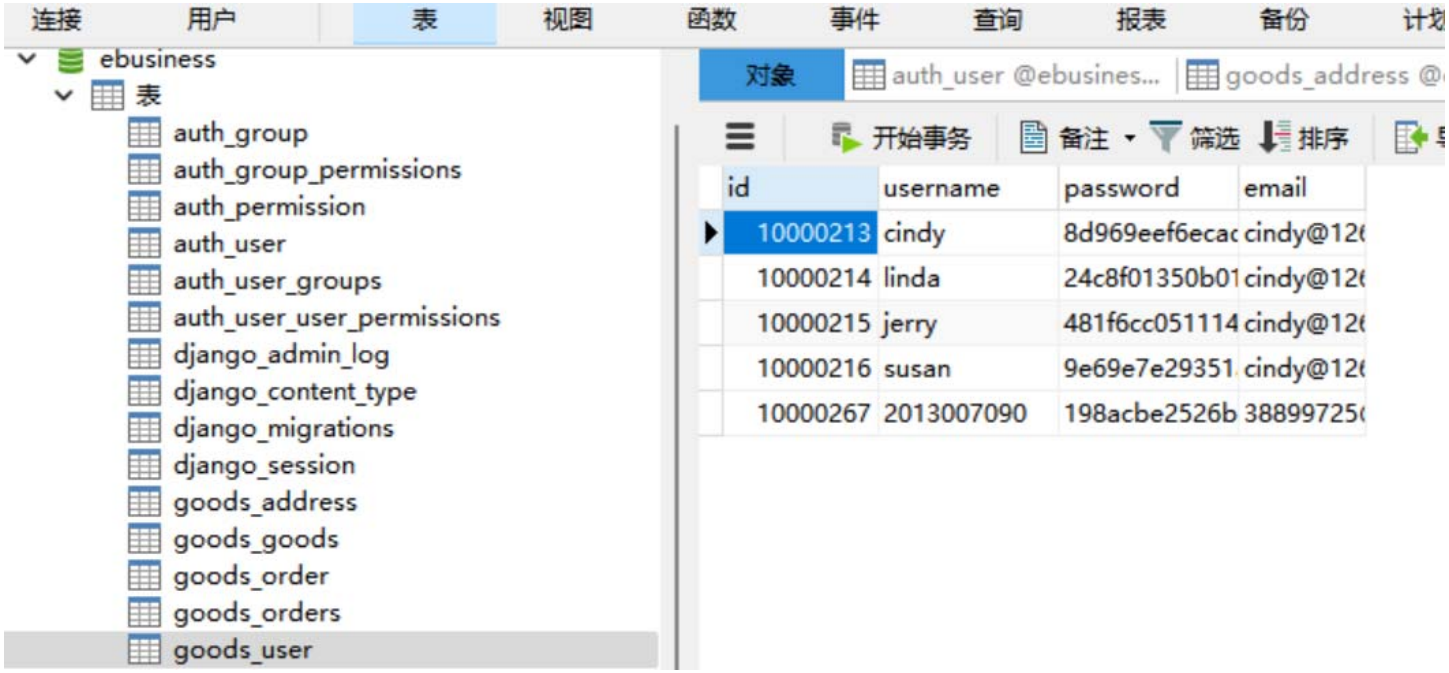
https://github.com/xianggu625/ebussiness





1、测试系统配置

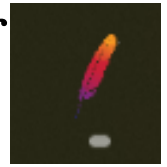
2、配置Mysql环境，将数据迁移到Mysql上，代码读写也进行转换



2、配置jmeter及jar

- 运行jmeter

D:\program files\Java\apache-jmeter-5.4.3\bin\jmeter



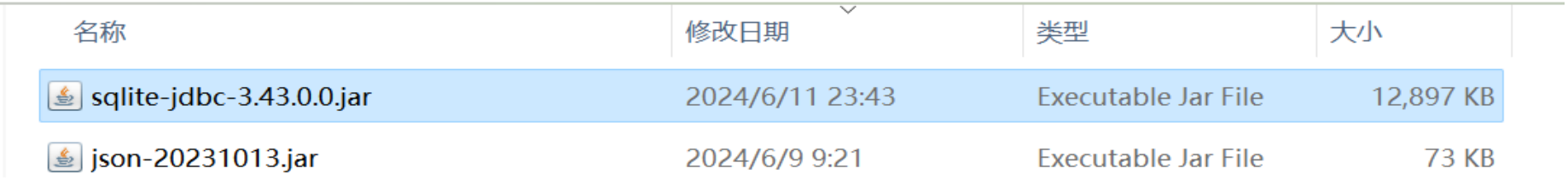
出错方法解决：

jmeter在非GUI中生成html报告及解决方法：

https://blog.csdn.net/weixin_43330588/article/details/127388521

jenkins执行jmeter时，报Begin size 1 is not equal to fixed size 5

https://blog.csdn.net/huashao0602/article/details/131984625



熟悉Jmeter性能测试元件



测试计划：使用jmeter测试的起点，是其他测试元件的容器，一个完整的测试计划包括多个线程组，逻辑控制器，取样器，监听器，配置元件。

（1）线程组：测试里每个任务都需要线程去处理，所有任务必须在线程组下面创建。

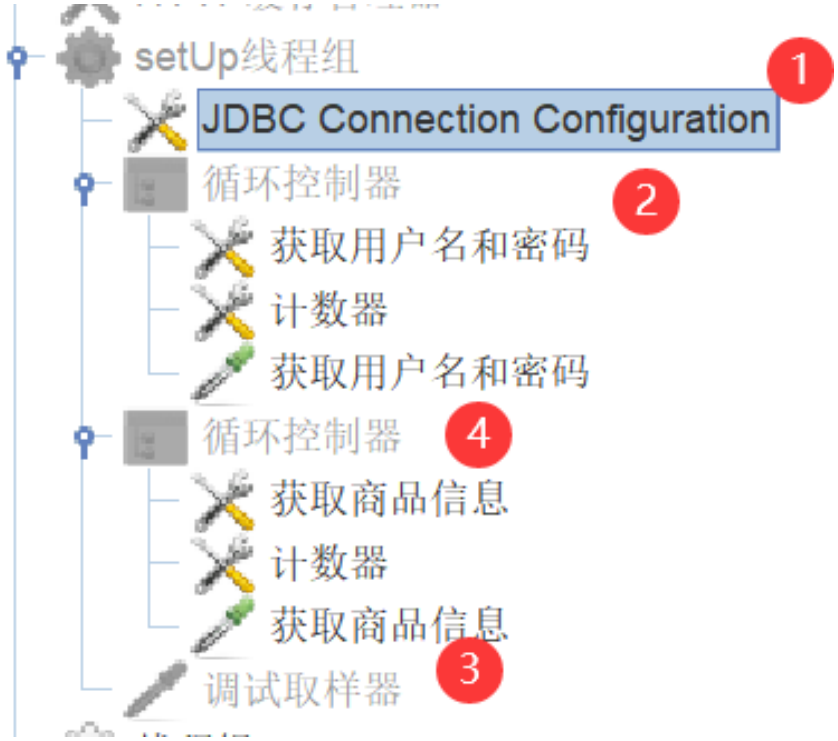
（2）setUp线程组

在做接口测试前，一般会做一些准备工作，比如数据准备、登录等。

（3）tearDown线程组

在做接口测试后，一般会做一些收尾工作，比如退出登录、关闭数据库连接、清理数据等。

熟悉Jmeter性能测试元件

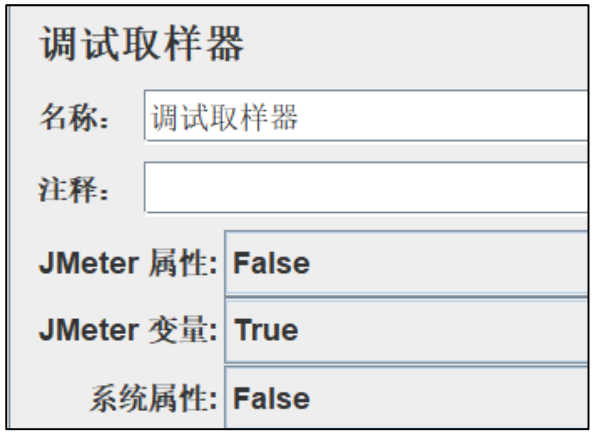


（1）数据库连接

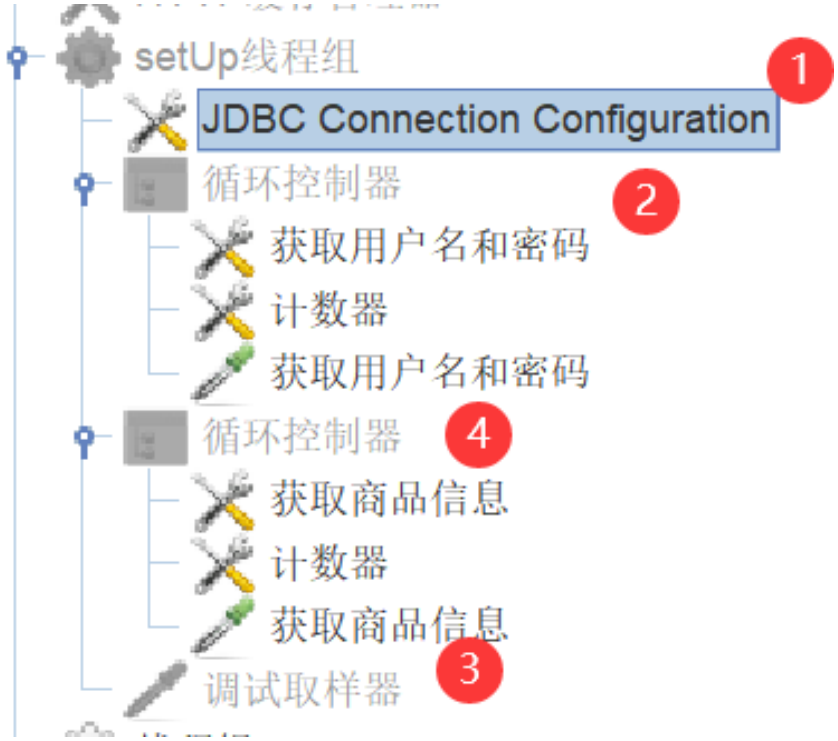
（2）设置用户连接

（3）设置商品连接

（4）调试取样器



熟悉Jmeter性能测试元件



（1）数据库连接

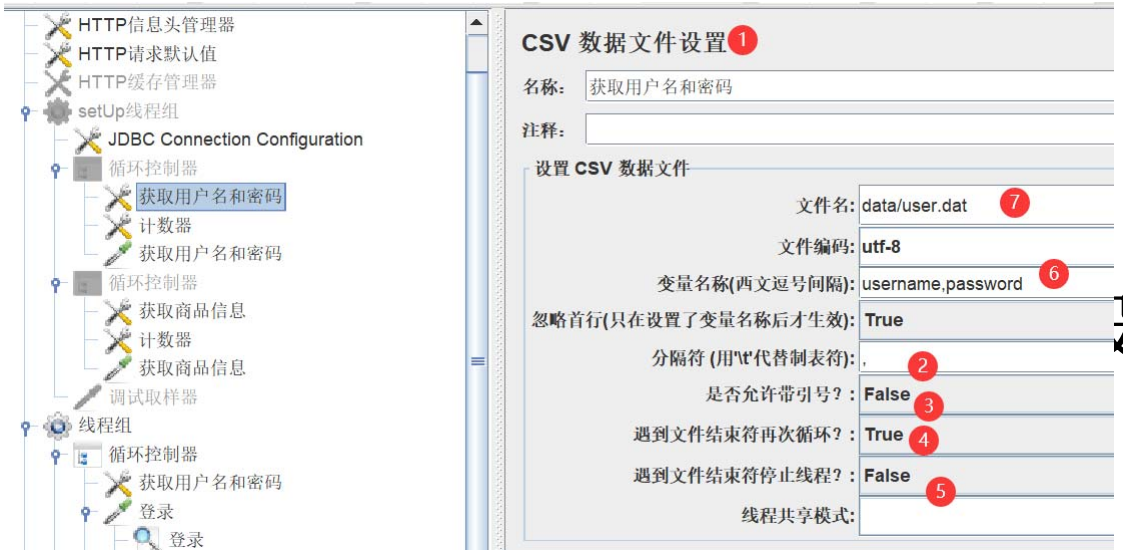
（2）设置用户连接

（3）设置商品连接

（4）调试取样器

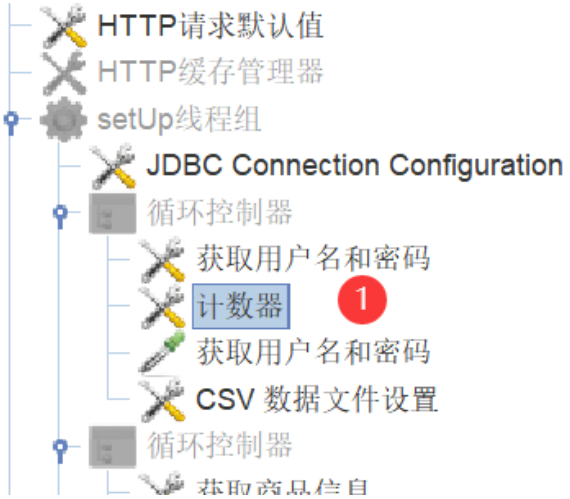
Setup线程

CSV 数据文件设置：从外部导入测试数据，相当于数据参数化通过从文件中导入大量的测试数据，来模拟大量真实用户发送并发请求



含义
文件名
文件编码
• 变量名称 • 多个变量用 , 分隔
• 忽略首行 • 只在设置了变量名称后才生效
• 分隔符 • 默认 ,
是否允许带引号
遇到文件结束符EOF 后再次循环
遇到文件结束符EOF 后停止运行线程?
线程共享模式

Setup线程





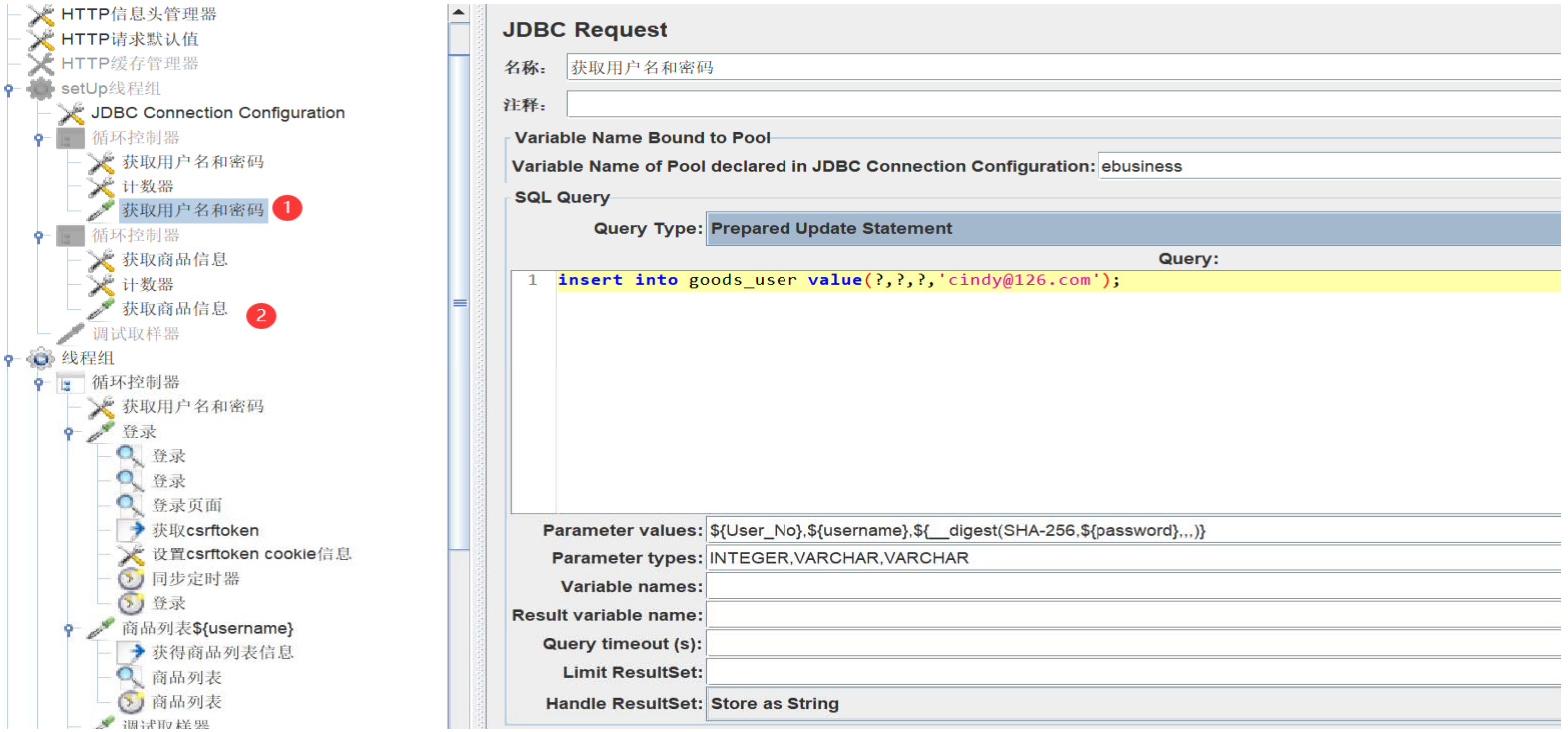
开始值（Starting value）：给定计数器的起始值、初始值，第一次迭代时，会把该值赋给计数器。

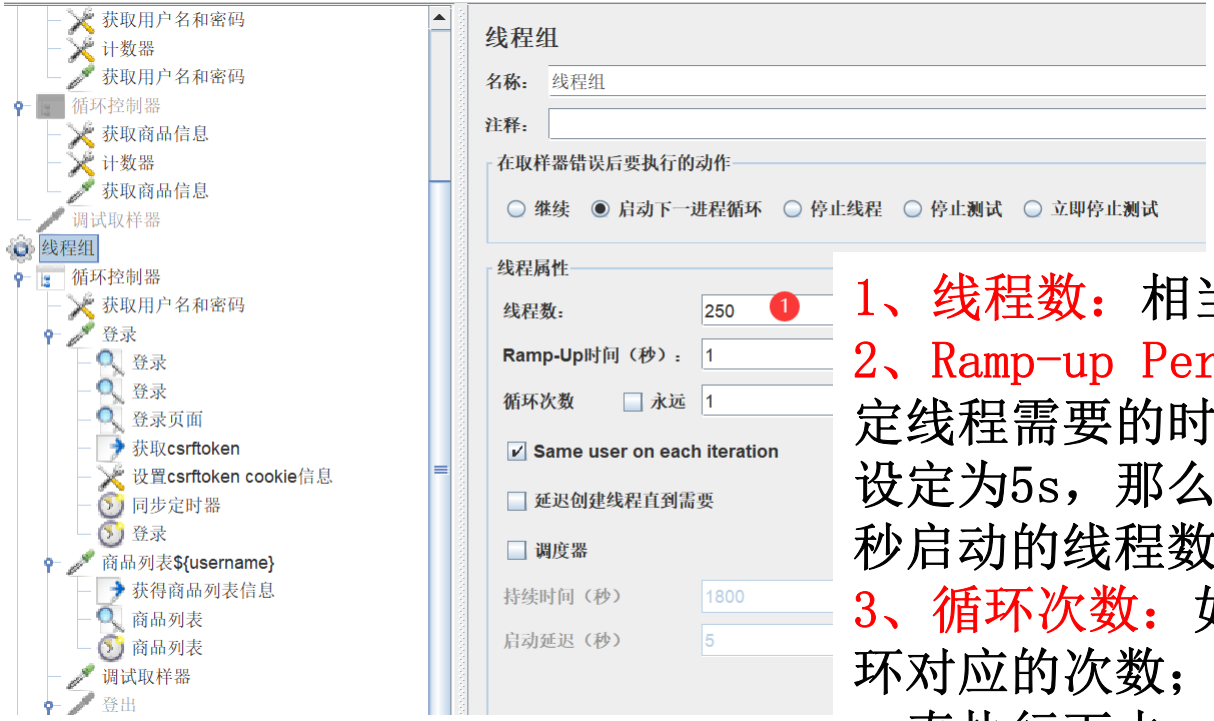
递增（Increment）：每次迭代后，给计数器增加的值。

引用名称（Exported Variable Name）：用于控制在其它元素中引用该值，比如：变量名称为reference_name，形式：${reference_name}。

JDBC request

配合JDBC Connection Configuration元件，使用Jmeter 对数据库进行访问



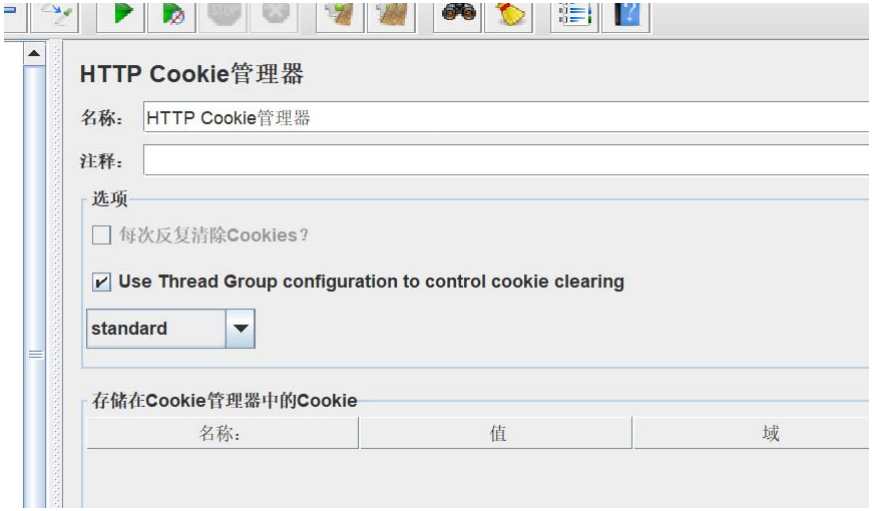


1、线程数：相当于模拟的用户数量；

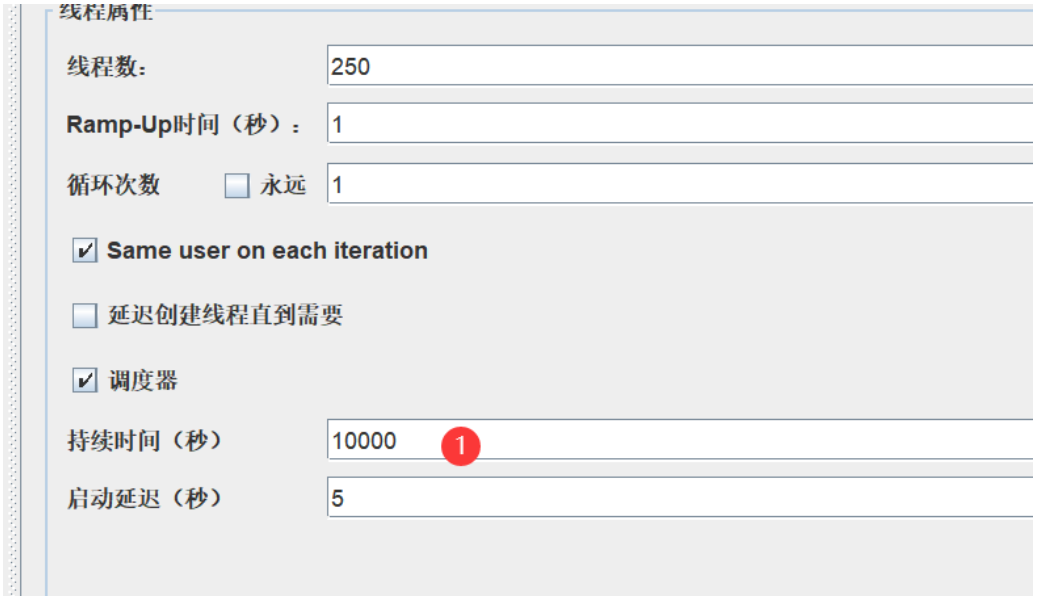
2、Ramp-up Period(in seconds)：达到指定线程需要的时间，例如线程数为10，时间设定为5s，那么就是5s加载 10个线程，每秒启动的线程数=10/5=2；

3、循环次数：如果填具体的数值，就是循环对应的次数；如果选择“Forever”，则一直执行下去，直到手动停止；4、Delay Thread creation until needed：延迟线程创建，直到需要才创建。

HTTP Cookie管理器



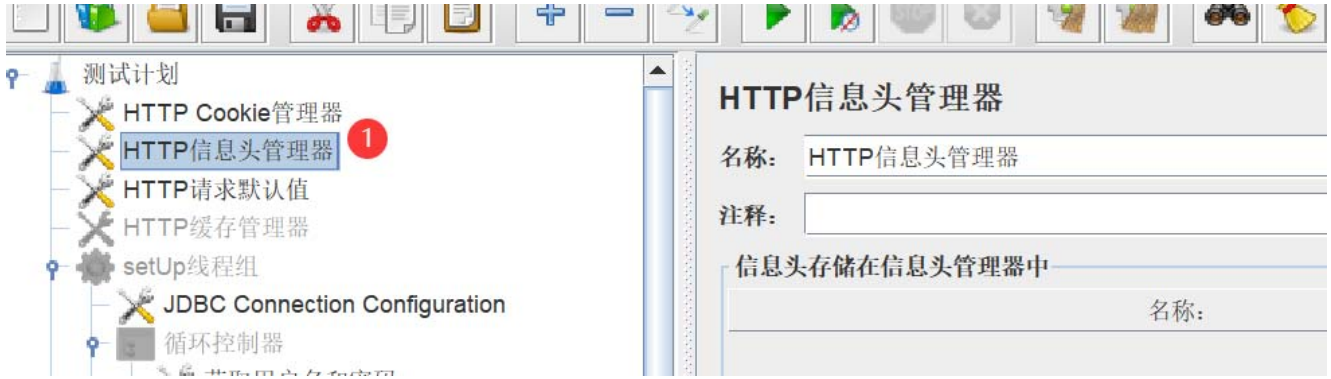
字段说明
控件名称
每次循环都清除 Cookie?
使用线程组配置项来控制清除 Cookie
管理 Cookie 的策略，默认就行
用户自定义 Cookie



持续时间：设置持续600（秒），表示这个线程组持续执行600s

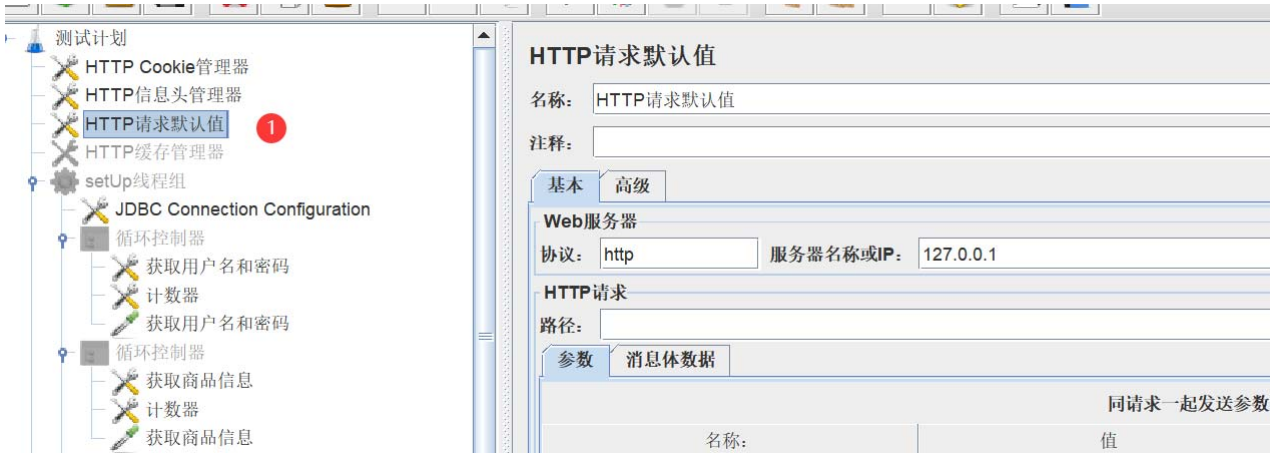
启动延迟：设置启动延迟为10。启动脚本后，在10s内不启动线程，10s过后，再在1s内启动完50个线程。启动延迟没有太大的意义

http信息头管理器



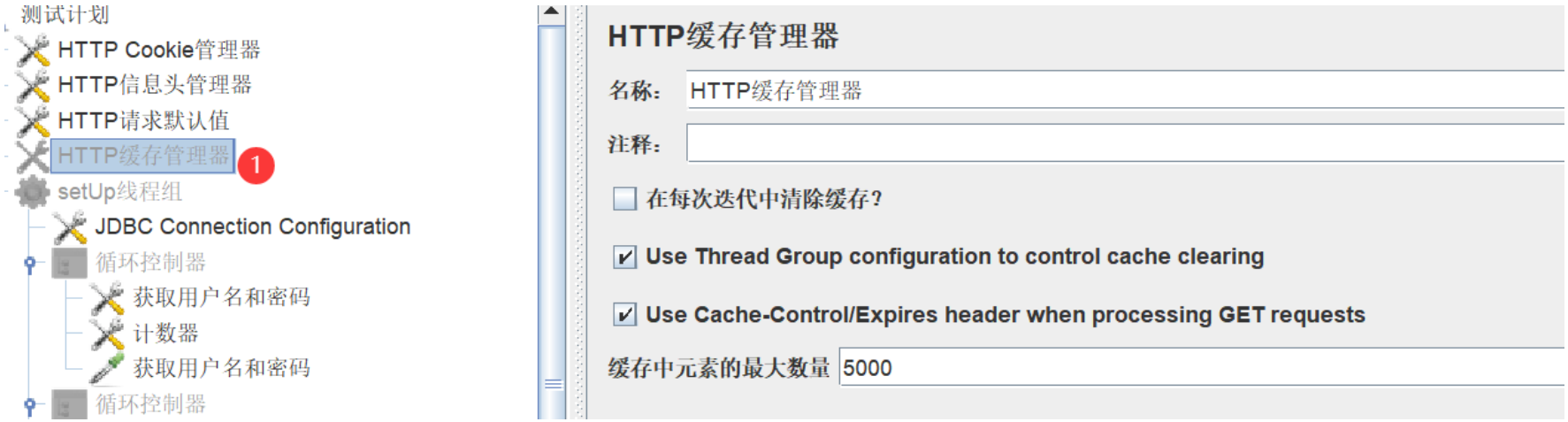
- 请求在发送时都会一般都会携带请求头信息，常见的包括请求的内容类型、token信息等，如果每个请求都要添加一个请求头信息，不易维护，可在测试计划节点下添加HTTP信息头管理器统一管理。

http请求默认值



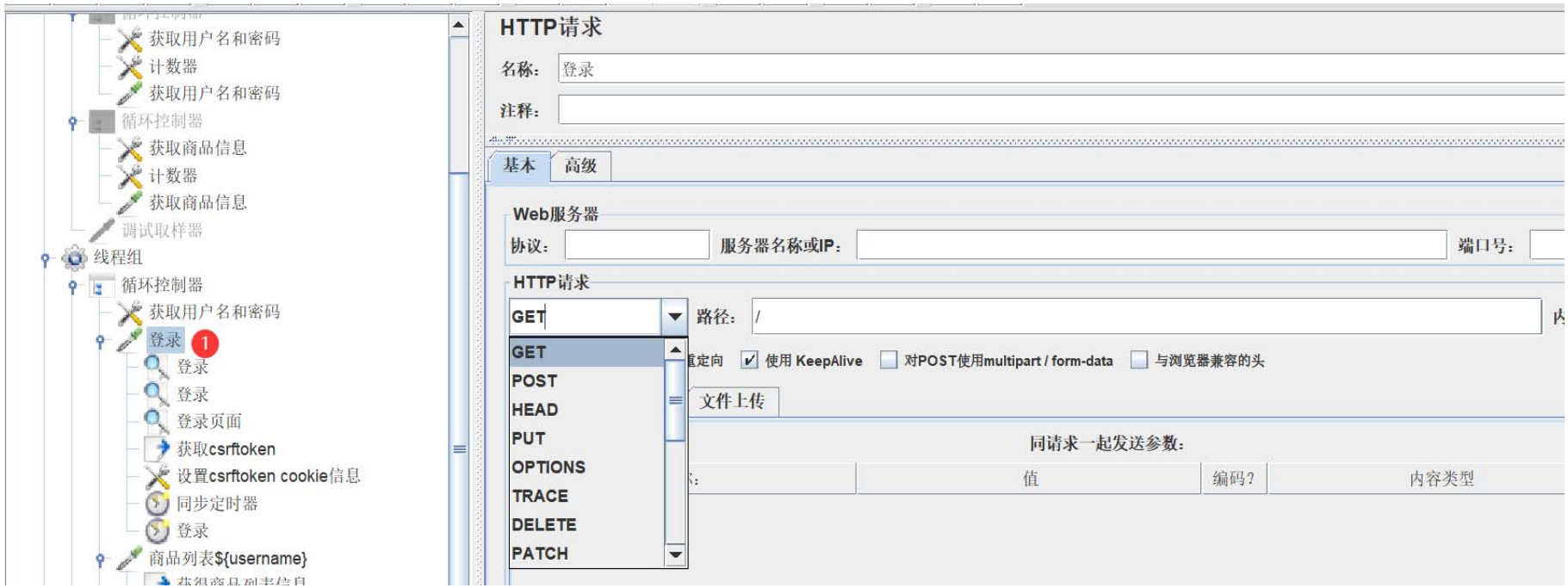
HTTP请求默认值：测试过程中有多个sampler（取样器）请求时，其中大多数如ip,协议，端口号，编码这些都是重复的，为了以后迁移或修改方便，将相同的参数放到一块。

http缓存管理器



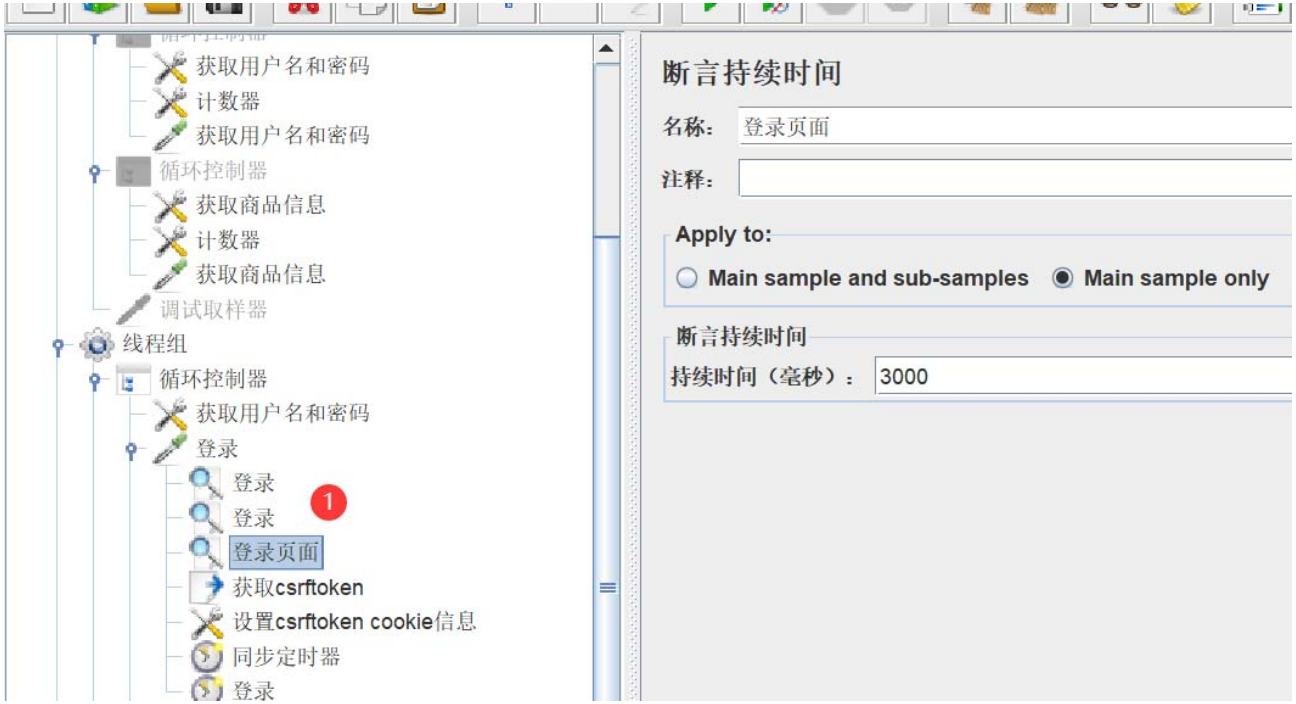
当使用浏览器第一次访问网页时， 浏览器会把一些资源文件(包括html, files, CSS Style sheets, JavaScript 和 multi-media 多媒体文件等)下载到本地缓存中， 当再次访问该页面时，浏览器就检查更新的内容， 并只下载那些未存储在缓存中的文件，已经缓存的资源就从本地磁盘中获取，这种机制减少了带宽的使用，有助于更快的加载网页。

HTTP 请求



GET：传输当前请求中描述的目标资源　POST：对请求负载中的指定资源进行处理

断言

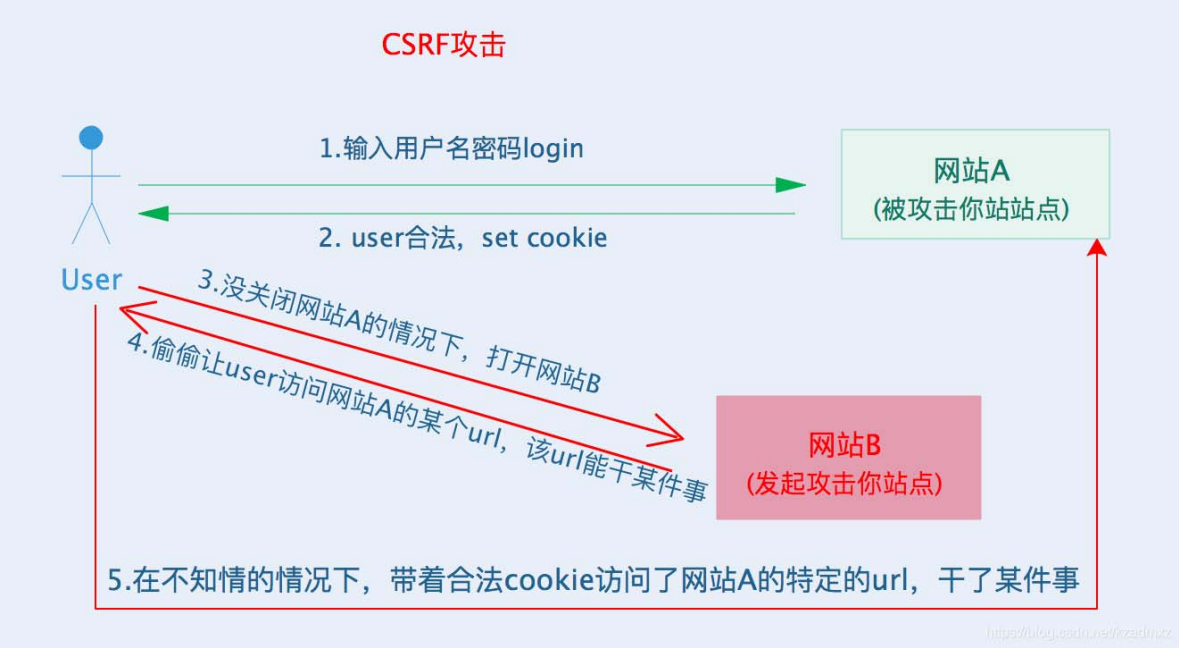


（1）响应断言

（2 beanshell断言

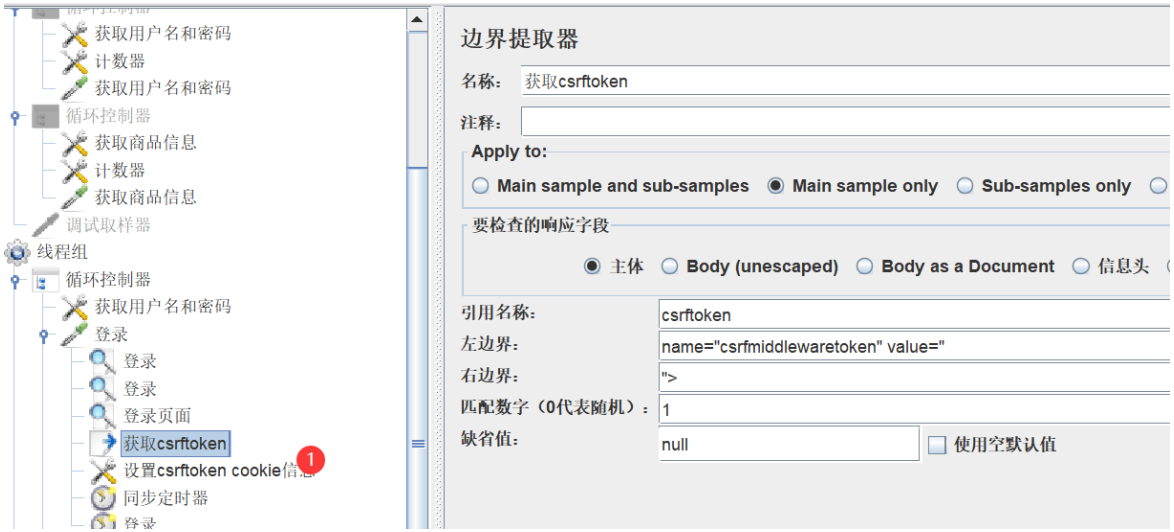
（3）断言持续时间

CSRF token



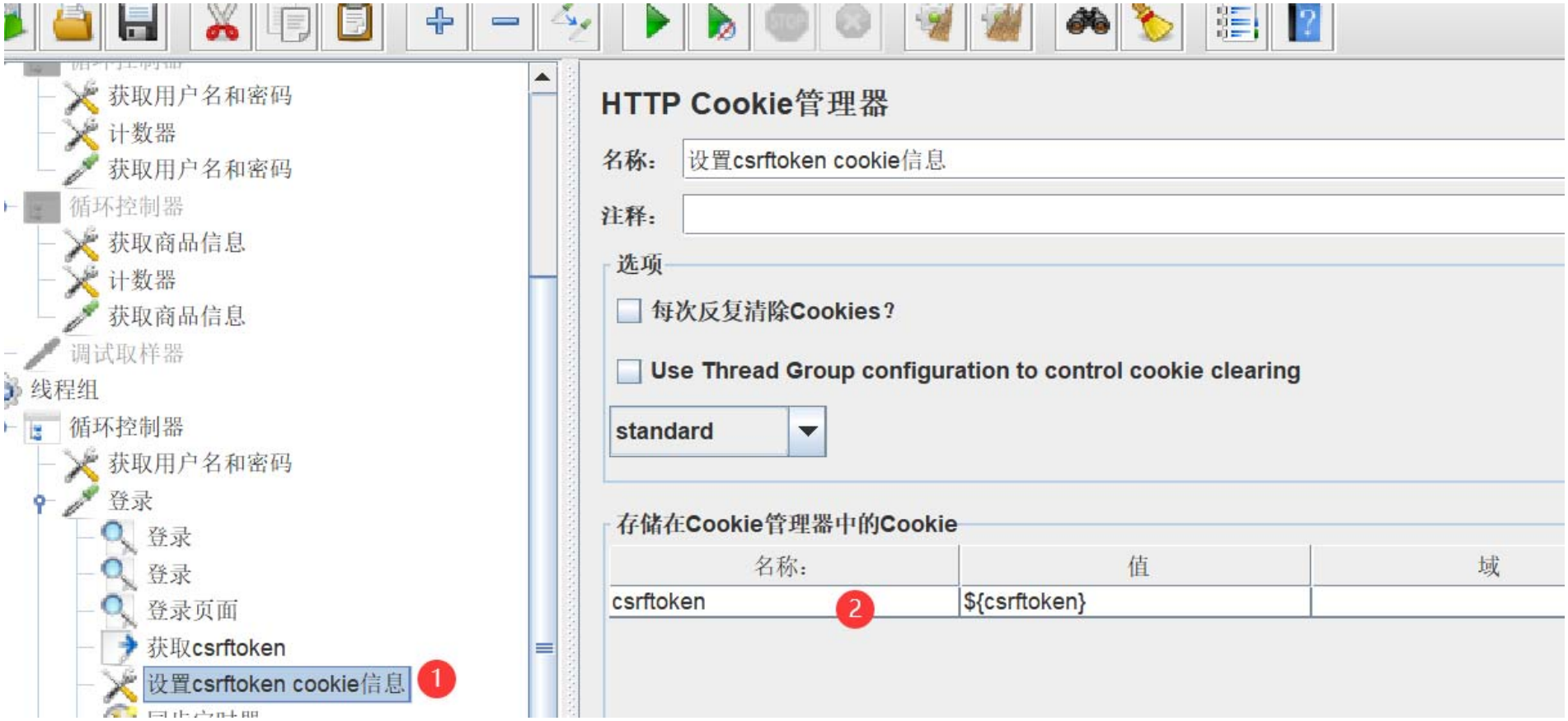
CSRF（Cross-site Request Forgery）跨站请求伪造（或者缩写为XSRF），也被称为"One Click Attack"或"Session Riding"（曾被列为互联网 20 大安全隐患之一），是一种借助社工对网站身份的恶意利用。

边界提取器

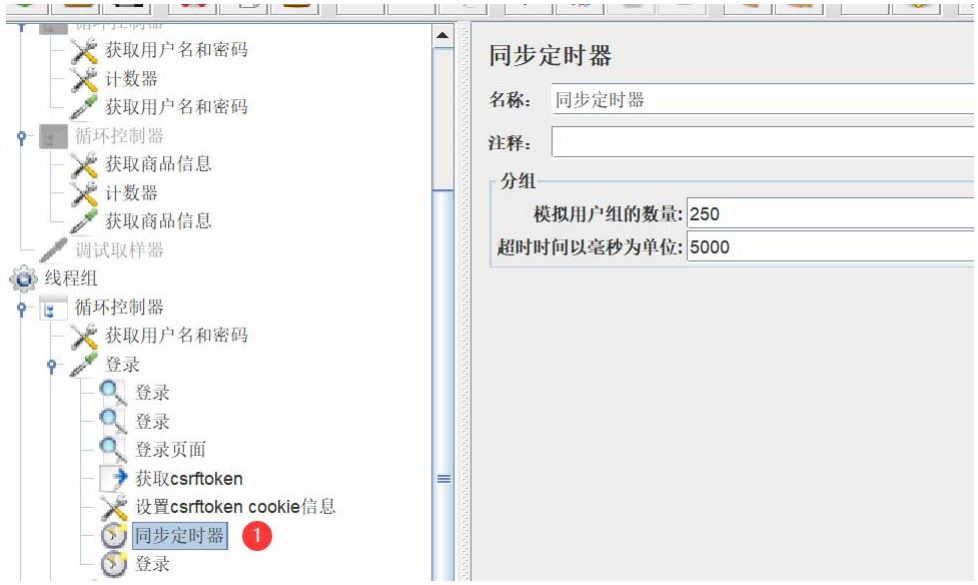


组件路径：HTTP请求->右键添加->后置处理器->边界提取器

用途：边界提取器(Boundary Extractor)是放在Sample请求之后执行的，用于获取左右边界中间的所有字符（不包含边界）

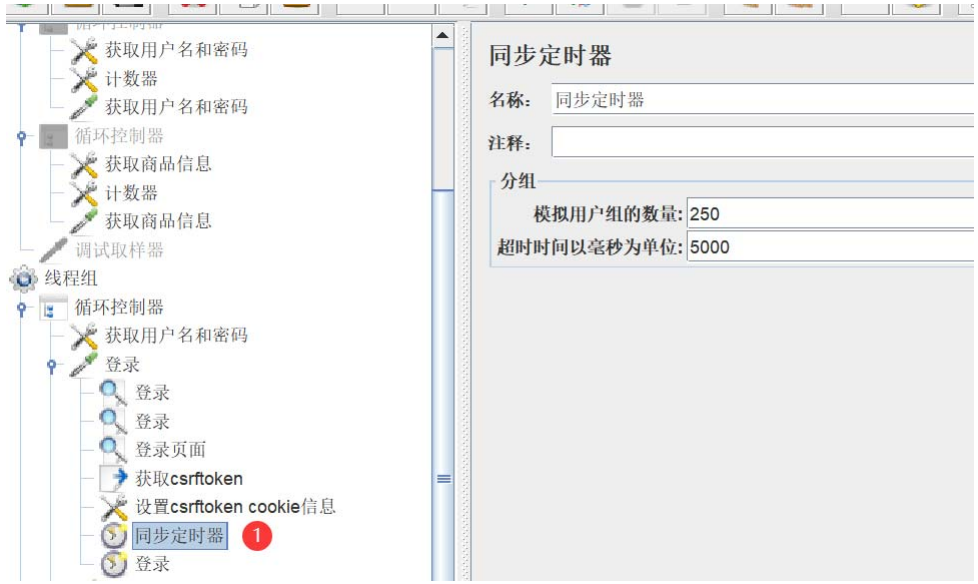

获取用户名和密码
计数器
获取用户名和密码
循环控制器
获取商品信息
计数器
获取商品信息
调试取样器
线程组
循环控制器
获取用户名和密码
登录
登录
登录
登录页面
获取csrftoken
设置csrftoken cookie信息
1
HTTP Cookie管理器
名称:
设置csrftoken cookie信息
注释:
选项
每次反复清除Cookies?
Use Thread Group configuration to control cookie clearing
standard
存储在Cookie管理器中的Cookie
名称:
值
域
csrftoken
2
${csrftoken}

同步定时器



当线程数量达到指定值后，再一起释放，可以瞬间产生很大的压力。

同步定时器



同步计时器，用来模拟多用户并发，或者说更严格的并发场景。

用来设置集合点，阻塞线程，同步虚拟用户，直到指定的线程数量到达后，恰好在同一时刻执行任务，再一起释放，可以瞬间产生很大的压力。
注意：一般情况，并发用户数多少，同步定时器就写多少。（多少并发就要等多少个人到齐一起释放）

统一随机定时器（Uniform Random Timer）

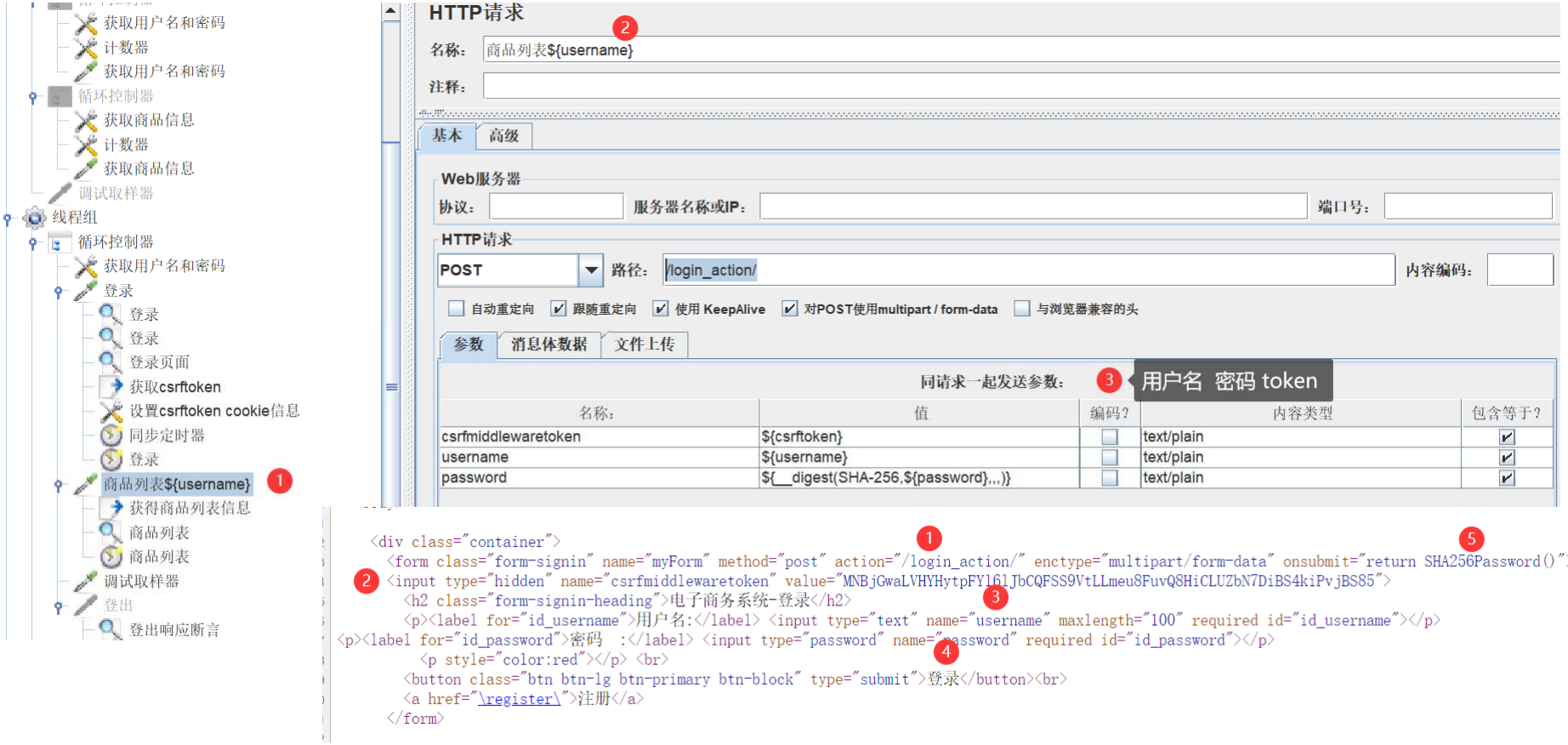


Random Delay Range（随机延迟范围）:它定义了随机延迟的范围。例如设置为500，表示随机延迟会在0到500毫秒之间取值。最终的延迟时间是固定延迟偏移量加上这个随机取值。

Constant Delay Offset（固定延迟偏移量）：这是一个固定的延迟值，单位为毫秒。例如设置为1000，表示在随机延迟的基础上至少会有1000毫秒（1秒）的延迟。

总延迟=Constant Delay Offset（固定延迟偏移量）+Random Delay Range（随机延迟范围）

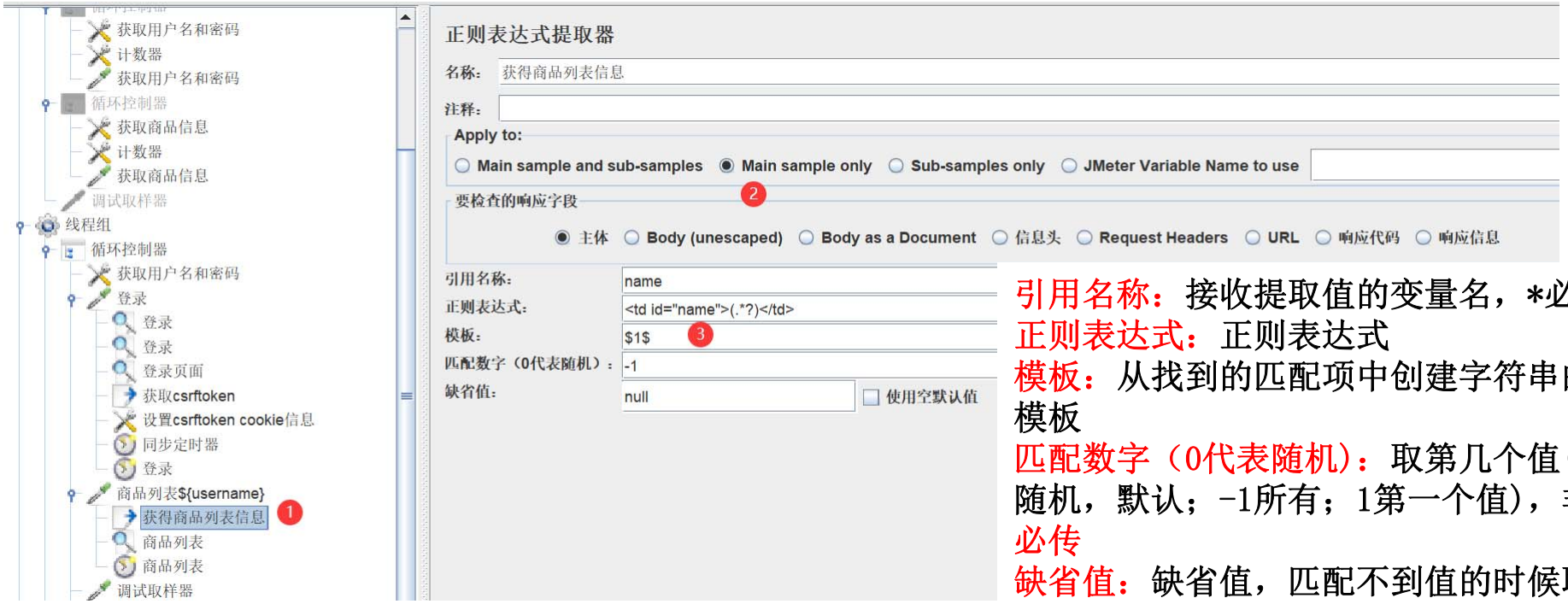
Post HTTP请求


获取用户名和密码
计数器
获取用户名和密码
循环控制器
获取商品信息
计数器
获取商品信息
调试取样器
线程组
循环控制器
获取用户名和密码
登录
登录
登录
登录页面
获取csrftoken
设置csrftoken cookie信息
同步定时器
登录
商品列表${username}
1
获得商品列表信息
商品列表
商品列表
调试取样器
登出
登出响应断言
HTTP请求
2
名称: 商品列表${username}
注释:
基本
高级
Web服务器
协议:
服务器名称或IP:
端口号:
HTTP请求
POST
路径: /login_action/
内容编码:
自动重定向
跟随重定向
使用 KeepAlive
对POST使用multipart / form-data
与浏览器兼容的头
参数
消息体数据
文件上传
同请求一起发送参数:
3
用户名 密码 token
名称:
值
编码?
内容类型
包含等于?
csrfmiddlewaretoken
${csrftoken}
text/plain
username
${username}
text/plain
password
${__digest(SHA-256,${password},,,)}
text/plain
1
5
2
3
4

<form class="form-signin" name="myForm" method="post" action="/login_action/" enctype="multipart/form-data" onsubmit="return SHA256Password()">
<input type="hidden" name="csrfmiddlewaretoken" value="MNBjGwaLVHYHytpFYl6lJbCQFSS9VtLLmeu8FuvQ8HiCLUZbN7DiBS4kiPvjBS85">
<h2 class="form-signin-heading">电子商务系统-登录</h2>
<label for="id_username">用户名:</label> <input type="text" name="username" maxlength="100" required id="id_username">
<label for="id_password">密码 :</label> <input type="password" name="password" required id="id_password">

<button class="btn btn-lg btn-primary btn-block" type="submit">登录</button>
注册
</form>

正则表达式提取器



引用名称：接收提取值的变量名，*必传

正则表达式：正则表达式

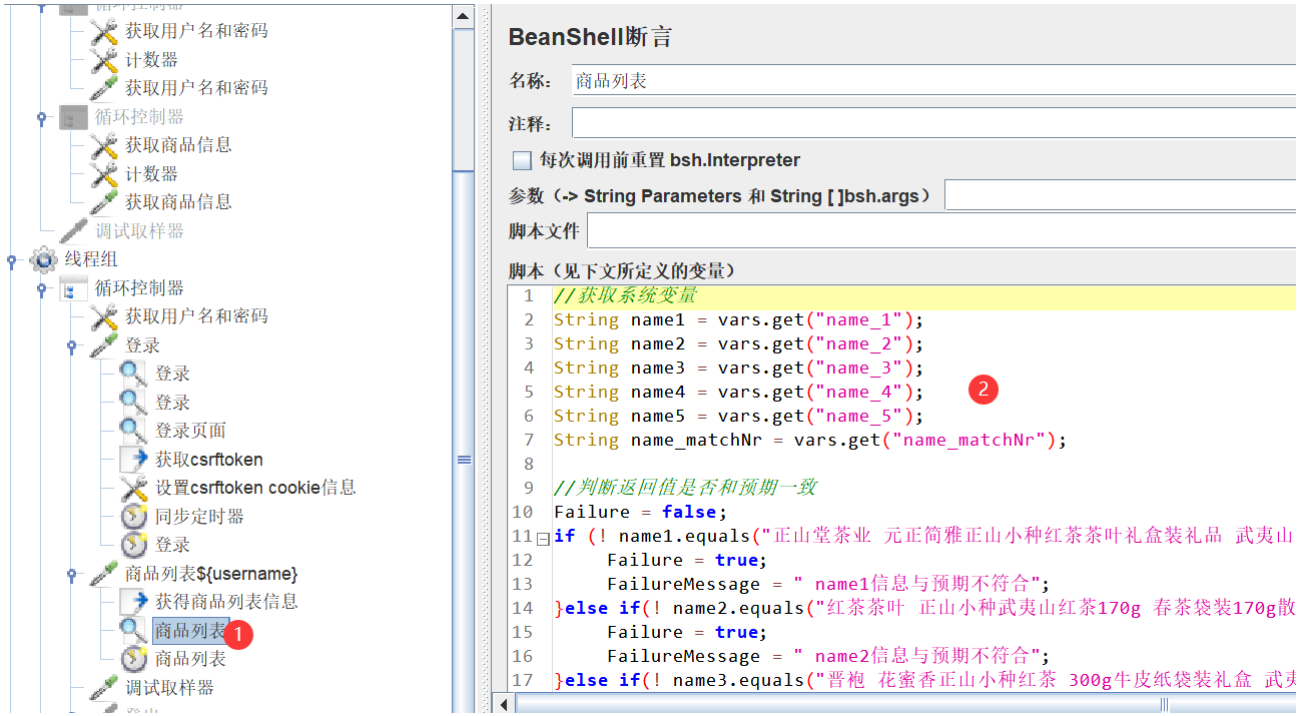
模板：从找到的匹配项中创建字符串的模板

匹配数字（0代表随机）：取第几个值(0:随机，默认；-1所有；1第一个值)，非必传

缺省值：缺省值，匹配不到值的时候取该值，非必传

使用空的默认值：勾选后，提取不到值时，则返回空字符串

Beanshell 断言

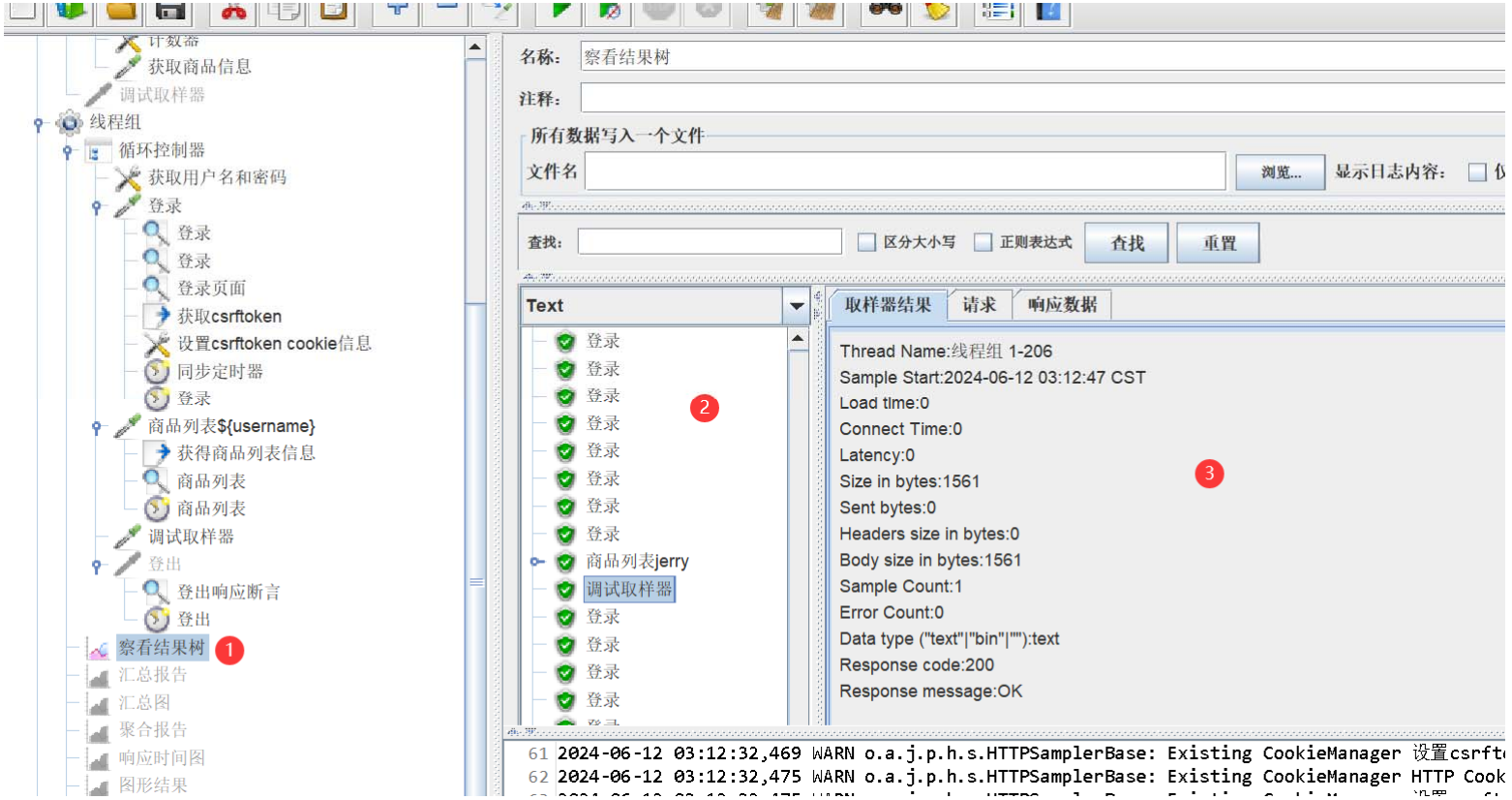


BEANSHELL

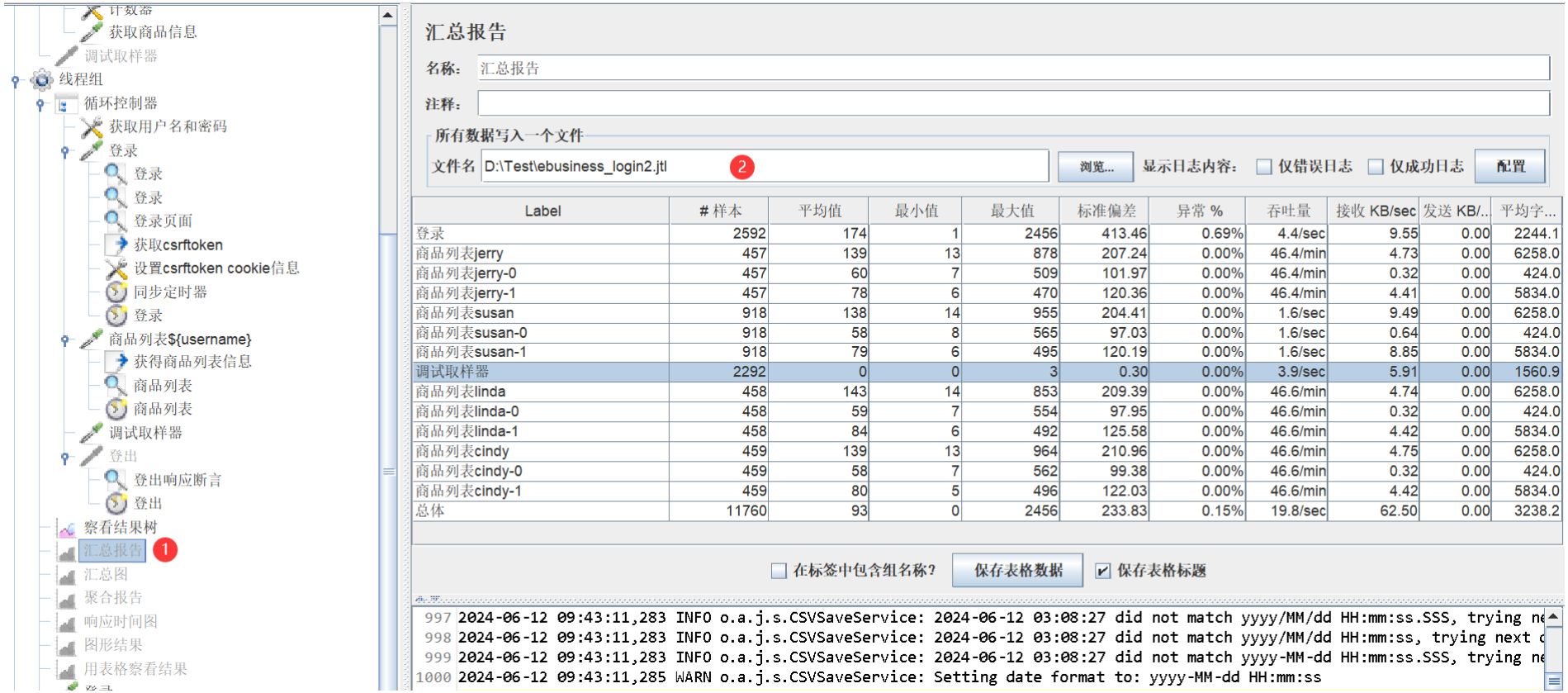


Beanshell是一种类似**JAVA**的脚本语言，通过**BeanShell**可以对请求数据、响应数据或环境变量进行更加灵活的处理和判断。还可以直接调用外部的**JAR**包，例如：可以直接引入现成的第三方**JSON**解析

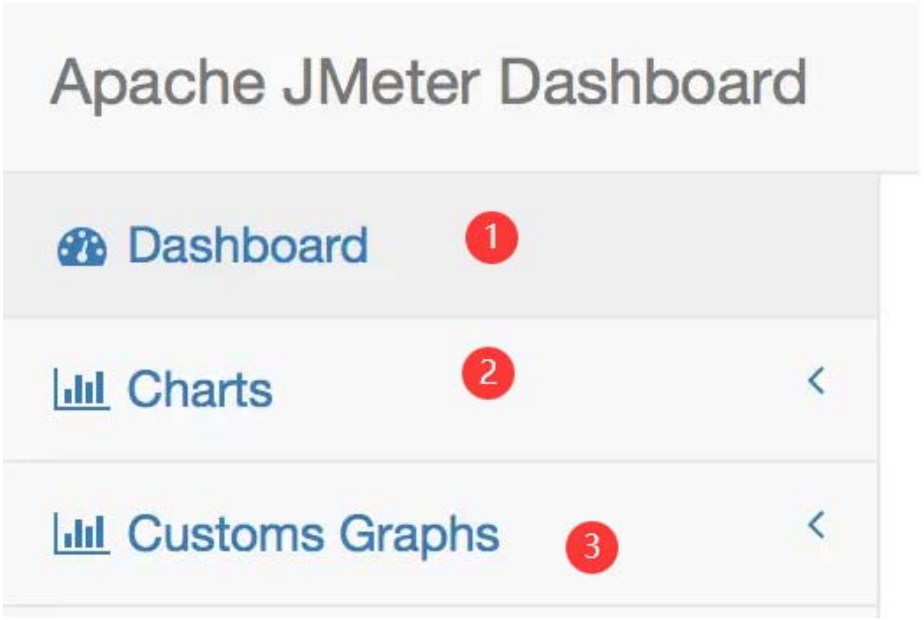
查看结果树


获取商品信息
调试取样器
线程组
循环控制器
获取用户名和密码
登录
登录
登录
登录页面
获取csrftoken
设置csrftoken cookie信息
同步定时器
登录
商品列表${username}
获得商品列表信息
商品列表
商品列表
调试取样器
登出
登出响应断言
登出
察看结果树
1
汇总报告
汇总图
聚合报告
响应时间图
图形结果
名称: 察看结果树
注释:
所有数据写入一个文件
文件名
浏览...
显示日志内容:
查找:
区分大小写
正则表达式
查找
重置
Text
登录
登录
登录
登录
登录
登录
登录
登录
商品列表jerry
调试取样器
登录
登录
登录
登录
2
取样器结果
请求
响应数据
Thread Name:线程组 1-206
Sample Start:2024-06-12 03:12:47 CST
Load time:0
Connect Time:0
Latency:0
Size in bytes:1561
Sent bytes:0
Headers size in bytes:0
Body size in bytes:1561
Sample Count:1
Error Count:0
Data type ("text"|"bin"|""):text
Response code:200
Response message:OK
3
61 2024-06-12 03:12:32,469 WARN o.a.j.p.h.s.HTTPSamplerBase: Existing CookieManager 设置csrft
62 2024-06-12 03:12:32,475 WARN o.a.j.p.h.s.HTTPSamplerBase: Existing CookieManager HTTP Cook

监听器


计数器
获取商品信息
调试取样器
线程组
循环控制器
获取用户名和密码
登录
登录
登录
登录页面
获取csrftoken
设置csrftoken cookie信息
同步定时器
登录
商品列表${username}
获得商品列表信息
商品列表
商品列表
调试取样器
登出
登出响应断言
登出
察看结果树
汇总报告
1
汇总图
聚合报告
响应时间图
图形结果
用表格察看结果
汇总报告
名称: 汇总报告
注释:
所有数据写入一个文件
文件名 D:\Test\ebusiness_login2.jtl
2
浏览...
显示日志内容:
仅错误日志
仅成功日志
配置
Label # 样本 平均值 最小值 最大值 标准偏差 异常 % 吞吐量 接收 KB/sec 发送 KB/... 平均字...
登录 2592 174 1 2456 413.46 0.69% 4.4/sec 9.55 0.00 2244.1
商品列表jerry 457 139 13 878 207.24 0.00% 46.4/min 4.73 0.00 6258.0
商品列表jerry-0 457 60 7 509 101.97 0.00% 46.4/min 0.32 0.00 424.0
商品列表jerry-1 457 78 6 470 120.36 0.00% 46.4/min 4.41 0.00 5834.0
商品列表susan 918 138 14 955 204.41 0.00% 1.6/sec 9.49 0.00 6258.0
商品列表susan-0 918 58 8 565 97.03 0.00% 1.6/sec 0.64 0.00 424.0
商品列表susan-1 918 79 6 495 120.19 0.00% 1.6/sec 8.85 0.00 5834.0
调试取样器 2292 0 0 3 0.30 0.00% 3.9/sec 5.91 0.00 1560.9
商品列表linda 458 143 14 853 209.39 0.00% 46.6/min 4.74 0.00 6258.0
商品列表linda-0 458 59 7 554 97.95 0.00% 46.6/min 0.32 0.00 424.0
商品列表linda-1 458 84 6 492 125.58 0.00% 46.6/min 4.42 0.00 5834.0
商品列表cindy 459 139 13 964 210.96 0.00% 46.6/min 4.75 0.00 6258.0
商品列表cindy-0 459 58 7 562 99.38 0.00% 46.6/min 0.32 0.00 424.0
商品列表cindy-1 459 80 5 496 122.03 0.00% 46.6/min 4.42 0.00 5834.0
总体 11760 93 0 2456 233.83 0.15% 19.8/sec 62.50 0.00 3238.2
在标签中包含组名称?
保存表格数据
保存表格标题
997 2024-06-12 09:43:11,283 INFO o.a.j.s.CSVSaveService: 2024-06-12 03:08:27 did not match yyyy/MM/dd HH:mm:ss.SSS, trying ne
998 2024-06-12 09:43:11,283 INFO o.a.j.s.CSVSaveService: 2024-06-12 03:08:27 did not match yyyy/MM/dd HH:mm:ss, trying next c
999 2024-06-12 09:43:11,283 INFO o.a.j.s.CSVSaveService: 2024-06-12 03:08:27 did not match yyyy-MM-dd HH:mm:ss.SSS, trying ne
1000 2024-06-12 09:43:11,285 WARN o.a.j.s.CSVSaveService: Setting date format to: yyyy-MM-dd HH:mm:ss

jemeter报告解析



https://zhuanlan.zhihu.com/p/623270105

（1）Dashboard(基本报告情况)

（2）Charts（详细报告）

Customs Graphs 默认无信息，可以通过设置user.properties文